

Hybrid Implementation and Evaluation of Classical and Post-Quantum Cryptographic Mechanisms in AI-Based Web Services

A Project Report

Submitted by:

Himangi Tripathy (2241020030)

Nikita Tosh (2241020022)

Annanya Nayak (2241014058)

Reetu Kanungo (2241011008)

Rishi Raj Gupta (2241007054)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**Faculty of Engineering and Technology, Institute of Technical Education and
Research**

SIKSHA 'O' ANUSANDHAN (DEEMED TO BE) UNIVERSITY

Bhubaneswar, Odisha, India

(June 2026)



CERTIFICATE

This is to certify that the project report titled “Hybrid Implementation and Evaluation of Classical and Post-Quantum Cryptographic Mechanisms in AI-Based Web Services” being submitted by (Himangi Tripathy (2241020030), Nikita Tosh (2241020022), Annanya Nayak (2241014058), Reetu Kanungo (2241011008), Rishi Raj Gupta (2241007054)) to the Institute of Technical Education and Research, Siksha ‘O’ Anusandhan (Deemed to be) University, Bhubaneswar for the partial fulfillment for the degree of Bachelor of Technology in Computer Science and Engineering is a record of original confident work carried out by them under my/our supervision and guidance. The project work, in my/our opinion, has reached the requisite standard fulfilling the requirements for the degree of Bachelor of Technology.

The results contained in this project work have not been submitted in part or in full to any other University or Institute for the award of any degree or diploma.

(Name and signature of the Project Supervisor)

Department of Computer Science and Engineering

Faculty of Engineering and Technology;
Institute of Technical Education and Research;
Siksha ‘O’ Anusandhan (Deemed to be) University

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, Prof. Amit Kumar Kar, for his valuable guidance, continuous support, and encouragement throughout the completion of this project titled “Hybrid Implementation and Evaluation of Classical and Post-Quantum Cryptographic Mechanisms in AI-Based Web Services”.

We are also thankful to the Department of Computer Science and Engineering, Faculty of Engineering and Technology, Institute of Technical Education and Research, Siksha ‘O’ Anusandhan (Deemed to be) University, Bhubaneswar, for providing us with the necessary facilities and academic environment to successfully carry out this work.

Finally, we express our gratitude to everyone who directly or indirectly contributed to the successful completion of this project.

Place:

Signature of Students

Date:

DECLARATION

We declare that this written submission represents our ideas in our own words and where other's ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/fact/source in our submission. We understand that any violation of the above will cause for disciplinary action by the University and can also evoke penal action from the sources which have not been properly cited or from whom proper permission has not been taken when needed.

Signature of Students with Registration Numbers

Date: _____

REPORT APPROVAL

This project report titled “**Hybrid Implementation and Evaluation of Classical and Post-Quantum Cryptographic Mechanisms in AI-Based Web Services**” submitted by Himangi Tripathy (2241020030), Nikita Tosh (2241020022), Annanya Nayak (2241014058), Reetu Kanungo (2241011008), Rishi Raj Gupta (2241007054) is approved for the degree of *Bachelor of Technology in Computer Science and Engineering*.

Examiner(s)

Supervisor

Project Coordinator

PREFACE

The advent of large-scale quantum computing poses an existential threat for classical Public key infrastructures such as RSA and ECC via Shor’s algorithm. This has spurred a “Harvest Now, Decrypt Later” threat model, driving the move to NIST standardized Post-Quantum Cryptography (PQC). However, PQC algorithms like ML-KEM and ML-DSA have Larger key sizes and higher computational demands, which create a performance conflict for real-time AI-based web services with tight latency budgets. This project proposes an adaptive cryptographic framework that selects dynamically between classical, hybrid, and post- quantum encryption modes. The system employs a Reinforcement Learning agent, modeled as a contextual bandit, to maximize a multi-objective reward Function considering real-time CPU load, network conditions, and data sensitivity. The Framework implements per-sensitivity Double Q-tables to ensure that high-sensitivity Data is protected in a robust way, while minimizing overhead for low-risk requests. Using Liboqs and a fine-tuned DistilBERT sentiment analysis model, experimental results show that cryptographic overhead stays below 2.2 ms(milliseconds) even at high concurrency. The RL policy keeps a post quantum selection rate of over 75%, showing that the major bottleneck in Intelligent services is AI inference, not the optimized cryptographic layer. This research presents a practical pathway to achieve cryptographic agility and quantum-resilience in Modern digital infrastructures.

Keywords: Post-Quantum Cryptography · Reinforcement Learning · AI Web Services
· Cryptographic Agility · Hybrid Encryption

INDIVIDUAL CONTRIBUTIONS

Himangi Tripathy (2241020030)	Complete source code, including the initial implementation and all subsequent improvements made to the code. Experimentation section of the manuscript. Project report writing.
Nikita Tosh (2241020022)	Complete source code, including the initial implementation and all subsequent improvements made to the code. Proposed system section of the manuscript. Project report writing.
Annanya Nayak (2241014058)	Initial implementation of code and Abstract, Introduction, Literature survey, and Conclusion sections of the manuscript. Project report writing.
Reetu Kanungo (2241011008)	Project report writing and Literature paper review.
Rishi Raj Gupta (2241007054)	Prepared PowerPoint presentations and Literature paper review.

TABLE OF CONTENTS

	Title Page	i
	Certificate	ii
	Acknowledgement	iii
	Declaration	iv
	Report Approval	v
	Preface	vi
	Individual Contributions	vii
	Table of Contents	viii
	List of Figures	ix
	List of Tables	x
1.	INTRODUCTION	1
	1.1 Project Overview	1
	1.2 Motivations	1
	1.3 Uniqueness of the Work	3
	1.4 Report Layout	5
2.	LITERATURE SURVEY	6
	2.1 Review of Existing Systems / Related Work	6
	2.2 Research Gaps Identified	8
	2.3 Problem Identification	10
3.	MATERIALS AND METHODS	12
	3.1 Dataset(s) Description	12
	3.2 Schematic Layout/Model Diagram	14
	3.3 Methodologies Adopted	16
	3.4 Tools Used	22
	3.5 Experimental Setup	23
	3.5 Evaluation Metrics and Performance Measures	25
4.	RESULTS	27
	4.1 System Specification	27
	4.2 Parameters and Configuration Settings	28
	4.3 Experimental Results and Observations	31
	4.4 Performance Analysis	39
	4.5 Comparative Study and Discussion	41
5.	CONCLUSIONS	46
6.	REFERENCES	48
7.	APPENDICES	50
8.	REFLECTION OF THE TEAM MEMBERS ON THE PROJECT	55
9.	SIMILARITY REPORT	56

LIST OF FIGURES

No.	Figure Name	Page No.
1	System Architecture of the Adaptive Web Service	16
2	Fraction of Post-Quantum Algorithm Choices Under Four Levels of Load (HTTP)	33
3	Medium Sensitivity Q-Table Learning Curve 55 Concurrent Users 50 Request Rolling Average	34
4	Algorithm Selection Frequencies Over Time 55 Concurrent Users	35
5	CPU Usage at Decision Points 55 Concurrent Users, first 200 Decisions per Algorithm	36
6	Stacked Algorithm Distribution for Last 1,000 Decisions at Each Concurrency Level	37
7	Epsilon Decay Curve for Medium-Sensitivity Q-Table 55 Concurrent Users	37
8	Comparative End-to-End Latency: HTTP vs. HTTPS Across Four Load Levels	38

LIST OF TABLES

No.	Table Name	Page No.
1	Comparison of Related Works and Research Gap	9
2	State Space Discretisation for the Double Q-Learning Policy Engine	17
3	Illustrative Mapping from System State and Sensitivity to Expected Dominant Cryptographic Mode	19
4	REST API Endpoint Specification for the Adaptive AI Service	19
5	Tools and Libraries Used in the Adaptive AI System Implementation	22
6	Evaluation Metrics Used to Assess the Performance and Correctness of the Adaptive AI System	25
7	Cryptography module microbenchmark results	31
8	Aggregated load test results across four concurrency levels	32
9	Comparison of Post Quantum selection rate– previous vs current system.	42

1. Introduction

1.1. Project Overview

This project introduces an adaptive web service that combines real-time AI-driven sentiment Analysis with a dynamically configurable cryptographic protection layer. The system is Designed to address a fundamental limitation of current web service deployments where Cryptographic configurations are applied statically without regard to data sensitivity or real-Time server conditions. This static approach either wastes computational resources on Low-risk requests or fails to provide adequate protection for sensitive data under heavy Load.

The proposed framework accepts text input alongside a declared sensitivity level Low, medium, or high and selects a cryptographic mode for each request based on the Current system state. Three cryptographic modes are available: classical, hybrid, and post-quantum. The selection is made by a reinforcement learning policy engine that observes real-time CPU utilisation and response latency and learns over time which mode best balances security strength against computational overhead for each sensitivity tier.

The AI inference component employs a pre-trained DistilBERT model [18] for binary sentiment classification, executing on CPU to simulate a resource-constrained deployment environment. Post-quantum primitives are implemented using the Open Quantum Safe library liboqs (v0.15.0) [20], providing ML-KEM-768 [14] and ML DSA-44 [15] in accordance with NIST FIPS 203 and FIPS 204 standards respectively. The complete system was deployed using Docker Compose with Nginx providing TLS termination, and evaluated under simulated concurrent user traffic using the Locust load-testing framework.

1.2. Motivation

The security of today's modern digital communication relies heavily on classical public-key cryptosystems such as RSA, Diffie-Hellman, and Elliptic Curve Cryptography (ECC). These algorithms are secure because they rely on

computationally intensive mathematical problems that are difficult for current classical computers to solve [10]. However, the emergence of quantum computing can threaten the current encryption system such as Shor's algorithm can solve the mathematical problems underlying RSA and ECC in less time, rendering current system vulnerable to quantum attacks [11].

This has catalysed a "Harvest Now, Decrypt Model" threat model where attacker can store the encrypted data today and decrypt it later using Quantum Computers [4]. To solve this, the National Institute of Standards and Technology (NIST) started a global standardization process for post-quantum cryptographic algorithms. The First finalised standard FIPS 203 selected ML-KEM(derived from CRYSTALS-Kyber) as the main Key Encapsulation mechanism and other finalised algorithm for digital signature includes ML-DSA(Dilithium),Falcon and SPHINCS+. These algorithms use advanced mathematical structures like lattices and hash-based functions to secure against both classical and quantum attacks. While lattice based cryptosystems like ML-KEM performs efficiently using smaller keys, code-based alternatives like McEliece needs larger key sizes whilst providing better security [13].

.Even though PQC is more secure ,it presents new challenges. In protocols like TLS 1.3, PQC often requires increased computational resources and larger key sizes slowing down TLS handshakes. For example, powerful servers can implement PQC with minimum impact, but smaller devices or real-time systems become much slower, degrading the performance as some algorithms may require 12x more resources than others [2].

In a real time environment, such as AI-based web services any delay can negatively impact the user experience and service quality. Therefore, a hybrid cryptographic system combining both classical and post-quantum methods are recommended for backward compatibility and "defence-in-depth" during the transition process [3]. Recent research is using reinforcement learning (RL) and artificial intelligence (AI) to dynamically choose between encryption depending upon various performance parameters [2].

1.3. Uniqueness of the Work

Design and Implementation of an Adaptive Cryptographic Framework:

The project a modular four-layer architecture designed to dynamically apply cryptographic protection during real-time web service operations. Unlike traditional systems that use a single, "one-size-fits-all" encryption method, this framework intercepts every inbound request and evaluates its context before selecting a cryptographic mode.

- The system passes traffic through an Middleware layer (Fast API), A Reinforcement Learning (RL) Policy Engine, an AI Inference Core (DistilBERT), and a Cryptographic Dispatch Layer.
- The framework can select one of three interchangeable modules As requested: Classical (RSA-2048 + AES-256 GCM), Post-Quantum (ML-KEM-768 + ML-DSA-44), and Hybrid (parallel execution of both).
- The architecture ensures quantum resistance outside the transport Layer by encrypting sensitive AI inference results directly meaning the data is secure Against future quantum decryption (“Harvest Now, Decrypt Later”) even if a TLS tunnel is intercepted today.

Sensitivity-Parameterized Reward Function and the Efficiency Gap:

A key innovation is the design of a multi-objective reward function that explicitly Accounts for the efficiency gap inherent in static security policies. In static environments Resources are often wasted on low-risk data (unnecessary PQC latency), or high-risk data is Left under-protected during high server load to maintain speed.

- The Reward Mechanism: The utility of RL agent is calculated as $R = w_{sec} \cdot S(a) - w_{lat} \cdot latency_penalty - w_{cpu} \cdot utilization_penalty + \delta(a, sensitivity)$.
- Risk-Aware Adaptivity: The framework keeps independent Q-tables for low/ medium/high Sensitivity levels so that “cross-sensitivity contamination” is

avoided, i.e. the optimization Policy for monitoring tasks does not impact the security requirements for sensitive Transactions.

- **Security Enforcement:** To prevent the agent from trading security for speed under load a Weak Cryptography Penalty (-0.20) is assessed whenever a non post-quantum method is Chosen for high-sensitivity data.

Detailed Benchmarking of NIST PQC in Low-Latency Systems:

This work evaluates PQC algorithms standardized by NIST under high concurrency with Real AI workloads.

- **Cryptographic Microbenchmarks:** Tests show that the overall cryptographic overhead is Very low less than 2.2 ms for all modules. It is discovered that ML-KEM-768 Decryption is significantly faster (0.298 ms) than traditional RSA-2048 private key Operations (1.836 ms).
- **Load Testing :** The system was tested under stress with concurrent users(15 to 80) to find Breaking points. The cryptographic layer continued to perform efficiently, but the system Reached CPU saturation at 55 concurrent users, causing response times to jump from 1,300 ms to 8,200 ms.
- **Bottleneck Identification:** A key insight is that the main bottleneck for intelligent web Services is the neural network inference step and not the highly-optimized mathematical Operations of standardized post-quantum primitives .

Reducing Hybrid Encryption Slowdown via Adaptive AI:

The research shows that AI-enabled cryptographic agility is a promising approach for the Adoption of quantum-safe systems without sacrificing user experience.

- **Hybrid Overhead Mitigation:** Hybrid encryption provides “defense-in-depth” by combining Classical and quantum secrets but effectively doubles the cryptographic workload. This is Mitigated by the adaptive RL policy that cleverly offloads PQC selections to lighter modes Only when the server load is critical and the data is less sensitive.

- Consistent Security Posture: The agent achieved a post-quantum selection rate above 75% for all tested load levels which ensures high security even under stress conditions.
- Transport Flexibility: The project illustrates that although TLS (HTTPS) introduces marginal Latency at low loads its effect becomes negligible compared to the heavy AI inference load At high concurrency justifying the use of an adaptive application-layer security Policy.

1.4. Report Layout

Section 2 presents a literature survey related work and the gap identified. Section 3 describes the data description, system layout, methodology used, experimental setup and performance evaluation. Section 4 details system specification. Parameters configuration, results obtained, performance analysis and its comparative discussion. Section 5 provides concluding remark

2 Literature Survey

2.1 Review of Existing Systems / Related Work

Overview of Classical Cryptography:

Modern digital security infrastructure relies on classical public-key cryptosystems such as RSA, Diffie-Hellman, and Elliptic Curve Cryptography(ECC) as they take an impractical amount of time to be solved, specifically integer factorization and discrete logarithms [10]. Although current cryptographic systems are secure against classical computational attacks, the emergence of large scale quantum computers presents an existential crisis. Shor's algorithm demonstrates that a reliable quantum computer can solve these problems in polynomial time making current public key infrastructure(PKI) vulnerable [19]. This vulnerability has introduced a "harvest now, decrypt later" threat model. Here, the attacker can store the encrypted data today and decrypt it once quantum computers become available. Symmetric cryptography is less severely impacted as it relies on key length. For example, the Grover's algorithm only speeds up the brute force attack which can be solved by doubling the key size such as moving from AES-128 to AES-256 [5].

The biggest priority for global security stakeholders is replacing current cryptography with post quantum cryptography. Experts predict that by 2035 quantum hardware will make 2048-bit RSA standards obsolete. Thus, the industry is accelerating the standardization of post quantum algorithms [5].

Overview of Post-Quantum Cryptography:

Post-Quantum cryptography refers to algorithms designed to secure both classical and quantum adversaries. The National Institute of Standards and Technology(NIST) launched a global standardization process in 2016 to secure post quantum algorithms from different mathematical families. After several years of testing NIST selected some official PQC standards such as ML-KEM(derived from CRYSTALS-Kyber) for key

encapsulation and ML-DSA(Dilithium) for digital signatures [13]. NIST has also defines 5 security categories ranked in increasing order strength Level 1 security being equivalent to AES-128, Level 2 comparable to difficulty in finding similar hash value in SHA-256/SHA3-256, Level 3 equivalent to AES-192, Level 4 comparable to difficulty in finding similar hash value in SHA-384/SHA3-384, Level 5 security being equivalent to AES-256 [19].

Lattice-based cryptography is the most popular PQC approach using problems like Module Learning With Errors(MLWE). ML-KEM is favoured due to its relatively moderate key sizes and high execution speed, whereas code based cryptography like McEliece provides stronger security but suffers from larger public key sizes which can exceed 250KB [5]. BIKE and HQC are alternative code based PQC algorithms that offers a more balanced trade-off between performance and key sizes[3].Hash-based signatures such as SPHINCS+(SLH-DSA) provides stateless security without depending on factorization or discrete logarithm but have a larger signature which makes it slower and requires more computation [9].

Experiments show that Kyber can generate keys much faster than classical RSA. But it still introduces challenges in terms of bandwidth usage. Similarly, PQC implementation increases TLS handshake size but up to 7x which sometimes can lead to packet fragmentation and increased latency in network environment such as mobile networks, IoT devices etc [1]. Also, algorithm like Falcon(FN-DSA) produces compact signatures but are harder to implement securely against side-channel attacks [5]. These characteristics necessitate selection of algorithms based on requirements and constraints of the deployment environment such as device capability, bandwidth, security, etc [13].

Existing Hybrid Cryptographic Approaches:

Hybrid cryptography is recommended to ensure backward compatibility and defense-in-depth during transition to post-quantum encryption. It combines classical algorithms (e.g. ECDHE) with post-quantum schemes (e.g. Kyber) ,where they operate in parallel to assure key generation speed to be confined by delayed scheme [1]. And remains secure as long as one of the combined algorithm is uncompromised [5].

Trails by leading companies like Cloudflare and Google have demonstrated that hybrid operations(e.g. Kyber+X25519) though doubles the cryptographic workload they only add a marginal latency of approximately 0.3-0.4 ms giving both traditional and quantum security [3]. IETF has also proposed for hybrid key exchange in TLS1.3(draft-ietf-tls-hybrid-design) which defines procedure for integrating both classical and PQC algorithms.

Some advanced frameworks use multifactor authentication and Quantum Random Number Generators(QRNGs) to generate more unpredictable random keys and strengthen session security [24].

2.2 Research Gaps Identified

Current PQC research treats algorithms(e.g. ML-KEM and ML-DSA) separately from higher level application logic(such as system load, request sensitivity, and latency). While performance for PQC algorithms has already been studied ,there is still a lack of research for adapting them dynamically for real time environment taking the system state into consideration. Present PQC deployments often use static policies ignoring dynamic factor such as transaction risk, real time server workload, etc.. For example, using heavy PQC configuration(e.g. ML-KEM1024 and ML-DSA level 5) for even low risk transaction unnecessary increasing workload, handshake latency, memory usage and slowing down real-time payment processing [7].

Recent literature has explored Reinforcement Learning(RL) for adaptive encryption depending on operational condition but are focused on energy efficiency in Wireless Sensors Networks and IoT devices. For example, Premakumari et al. uses Q-learning to dynamically change encryption level based on operational conditions such as energy consumption, latency, security in constrained network [17]. And there is not established framework that uses Q-learning to dynamically choose between classical, hybrid, and PQC modes for AI-based web services using real time workload and risk-aware conditions [2, 7].

Table. 1. *Comparison of related works and research gap*

Study	Focus	Gap
Ali et al. (2025) [2]	Q-learning and MAML-based dynamic PQC algorithm selection for mobile devices based on CPU load, battery, network latency, and threat level	Restricted to mobile context; no classical mode included; no web service integration; no sensitivity-based reward weighting
Balakrishnan et al. (2026) [7]	RL-based adaptive PQC parameter selection combined with GNN fraud detection; uses RL to choose PQC parameter sets based on transaction risk	Fraud detection drives decisions; does not switch between classical, hybrid, and PQC modes; no sensitivity tiers; no AI inference latency
Zafar & Iqbal (2025) [24]	Hybrid TLS framework integrating code-based PQC with classical algorithms; dual-handshake mechanism with adaptive fallback negotiation	No runtime adaptation; cipher selection is policy-driven at configuration time; ignores real-time load and sensitivity
Abbasi et al. (2025) [1]	Cross-platform PQC benchmarking of Kyber, NTRU, BIKE, Dilithium, and Falcon; evaluates TLS 1.3 handshake overhead and concurrency scaling	Static algorithm assignments only; no RL-based or context-aware selection; no classification by sensitivity
Souvatzidaki & Limniotis (2025) [19]	PQC key exchange performance in TLS 1.3 under emulated network conditions; compares Kyber, HQC, and BIKE against classical ECDH	No adaptive mode switching; evaluates fixed configurations only; no sensitivity or load-aware decision mechanism

2.3 Problem Identification

The problem of critical performance and security conflict due to the transition from Classical to Post-Quantum Cryptography (PQC) in real-time high-velocity environments Such as AI-based web services, is identified in the research.

The Existential Quantum Threat:

Modern digital security relies on classical public-key cryptosystems such as RSA and Elliptic Curve Cryptography (ECC). These are vulnerable to Shor's algorithm which can Solve their underlying mathematical problems in polynomial time with a large scale Quantum computer [11] and This has resulted in the threat model of "Harvest Now, Decrypt Later" where adversaries are collecting and storing encrypted data today to decrypt it once Quantum hardware becomes available [4].

Performance and Resource Incompatibility:

While NIST-standardized PQC algorithms like ML-KEM (Kyber) and ML-DSA (Dilithium) can protect against quantum attack, they introduce operational overheads [13]:

- Computational Load: Some PQC algorithms require more processing power with operations being up to 12x more resource-intensive than classical one [1].
- Bandwidth Expansion: PQC requires much larger key and signature sizes for example, Kyber can increase TLS handshake sizes by up to 7x leading to packet fragmentation and latency increase in constrained networks [19].
- Latency Penalties: Adding 1 KB of data in a TLS handshake can increase total response time by 1.5% which is not acceptable for high-velocity transaction systems [6].

The Inefficiency of Static Security Policies:

- Current industry uses static cryptographic configurations and this approach creates an "efficiency gap"[2]:

- **Wasted Resources:** Low-risk data can increase unnecessary latency and CPU penalties from using heavy PQC [7].
- **Insufficient Protection:** High-sensitivity data may not receive proper protection if the system defaults to classical modes under heavy server load to maintain speed [7].

AI-Based Web Services and Security Concerns:

Real-time web services using Artificial Intelligence for high-velocity tasks operate under strict latency budgets. Integrating PQC into these environments is difficult as the user can degrade because of increase in processing time and memory usage, for example latency sensitive applications such as video, voice, or digital payments [7].

Experiments reveal that every addition of 1KB of data to TLS handshake increases response time by 1.5% [6]. Naive PQC implementation increases the handshake sizes by 7 times and is slower than pure ECDHE by approximately 22-55%, with BIKE showing the largest increase (55%) which can slow down the performance in a high-concurrency cloud environment. PQC creates challenges under congested or unreliable networks where handshake latency spikes when packet loss occurs. PQC with large message size such as BIKE, requires up to 20 IP packets to complete a TLS handshake in small MTU settings [1].

For AI services accessed via mobile devices, deploying PQC algorithms are very challenging. These challenges are due to hardware limitations, network volatility, and energy sensitivity which increases memory consumption (BIKE often showing 50% higher than Kyber) computational overhead, and transmission latency rapidly draining battery resources [2].

3 Materials and Methods

3.1 Dataset(s) Description

The Sentiment Analysis Dataset Stanford Sentiment Treebank (SST-2):

The AI model of the system performs binary sentiment classification. The AI inference component classifies text as either positive or negative sentiment. This task was selected because it represents a practical artificial intelligence application programming interface workload. In this situation user-submitted text is processed by a service to return a structured prediction. The execution sequence must complete with minimal latency to ensure operational sustainable in production systems.

The Stanford Sentiment Treebank (SST-2) [26] by Socher et al., was used as the base dataset and accessed via the Hugging Face Hub identifier sst2. The dataset consists of individual sentences from movie reviews collected from Rotten Tomatoes [26]. Each sentence is labelled as either positive or negative by human annotators. This results in 67,349 training and 872 validation samples. Since lengths, syntactic structures and vocabularies vary across the corpus the model is trained on realistic linguistic diversity rather than a limited input distribution.

To ensure realistic testing conditions no pre-processing pipeline was applied to the input text and the raw text from the JSON request body was passed directly to the DistilBERT tokenizer without the case normalization, stop word removal, stemming, or auxiliary cleaning steps. This design configuration was chosen deliberately as the implementation of a pre-processing stage would compress the measured inference latency and misrepresent the operational cost of serving the model under production API condition where input text displays natural variation. Tokenization was executed with parameters set to max_length=512 and truncation=True to maintain consistency with the original fine-tuning configuration. Also for input shorter than 512 tokens padding was skipped to avoid redundant computation.

As SST-2 represents a core component of the General Language Understanding Evaluation (GLUE) benchmark [25] the baseline accuracy of the model on this dataset has been extensively documented and benchmarked against alternative architectures. This established external validation allowed the inference component to be treated as a verified baseline performance standard. So, the research focus was mainly kept on the cryptographic and policy layers where the core architectural contributions are situated.

The Pre-trained Model — DistilBERT:

The model selected for inference is `distilbert-base-uncased-finetuned-sst-2-english` [18] which is available on the Hugging Face Model Hub. This architecture represents a compressed and fine-tuned variant of the original BERT model introduced by Devlin et al. [25]. The base BERT model was trained on large text corpora with masked language modelling and next-sentence prediction objectives to learn general language understanding capabilities. These broad linguistic capabilities were specialized for the binary sentiment classification task through subsequent fine-tuning on the SST-2 dataset.

Sanh et al. [18] created DistilBERT using knowledge distillation so that a smaller student model is trained to mimic the output behaviour of the bigger BERT-base teacher, rather than simply training from raw data. The result is a model with 6 transformer layers instead of 12 and roughly 66 million parameters instead of 110 million. Despite that reduction, DistilBERT keeps approximately 97% of BERT's GLUE performance, cuts model size by 40%, and reduces inference latency by around 60% [18]. Those numbers make it a practical fit for a CPU-bound API environment where every millisecond of inference time counts.

Inference runs with ``device=-1``, forcing CPU execution through HuggingFace Transformers' PyTorch backend with no GPU involved. This configuration is designed to imitate a resource-constrained server. If the cryptographic overhead is negligible even under this CPU-only constraint, the result applies equally well to better-equipped deployments, making it a conservative and credible lower bound for the system's performance.

The model is loaded once at startup and remains in memory to prevent initialization delays for incoming requests. Each request produces three outputs: the binary sentiment classification (POSITIVE or NEGATIVE), a confidence score between 0 and 1, and the execution latency in milliseconds.

3.2 Schematic Layout / Model Diagram

The Adaptive AI service organizes all processing into four modular layers, each of which has its own responsibility. Fig. 1 shows the complete system architecture. Every inbound HTTP request passes the layers in a defined sequence: API and middleware (Layer 2) → policy engine (Layer 4) → AI inference (Layer 1) → cryptographic layer (Layer 3), with the encrypted response going back to the user in reverse and it contains encrypted data.

Layer 2—API and Observability Middleware(FastAPI). API and Middleware is built using the FastAPI Framework. It handles all incoming and outgoing requests of the system. When a request hit the middleware it starts a high-resolution timer, increments a global request counter, and validates the JSON payload using Pydantic schema enforcement. The validated input is described by the schema `{"text": str, "sensitivity": "low"|"medium"|"high"}`. It also checks the current CPU utilisation percentage and tracks a rolling average of end-to-end latency. These state variables are passed to the policy engine prior to inference. On its way out, middleware records the total elapsed time, injects an X-Request-Time-Ms header into response and sends this latency reading back to the engine. This acts as a scalar reward signal closing the online learning loop.

Layer 4 —Policy Engine (Double Q-Learning). The policy engine observes the current system state and selects a cryptographic action before AI inference begins. It maintains three independent pairs of Q-tables, one pair for each sensitivity level (low, medium, high), and applies an ϵ -greedy exploration strategy. The selected mode classical, hybrid, or post_quantum is forwarded to the cryptographic dispatch table and stored alongside the observed state so that the reward received after request completion

can be attributed to the correct state–action pair. The detailed expression of the state space, action set, and reward function is given in Section 3.3.

Layer 1 — AI Inference Core (DistilBERT). The system uses a pretrained DistilBERT model that handles inference by classifying binary sentiment and reads input text from the requested payload and tokenizes it. Then the model processes this data on the CPU to simulate a resource-constrained deployment environment. The layer returns a structured result containing the predicted sentiment label (POSITIVE or NEGATIVE), the associated confidence score, and the measured inference latency in milliseconds.

Layer 3 — Cryptographic Layer. The Cryptographic layer takes the result from the model and serializes it a JSON string. The system encrypts this data using the module selected by the policy engine. This layer applies these cryptographic operations using 3 interchangeable modules:

- Classical mode: The System uses RSA-2048 with OAEP-SHA256 padding for asymmetric key transport and combines with AES-256-GCM for symmetric authenticated encryption.
- Post-quantum mode: This mode relies on ML-KEM-768 (Kyber-768, NIST FIPS 203 [14]) for key encapsulation and ML-DSA-44 (Dilithium3, NIST FIPS 204 [15]) to guarantee digital signature integrity. It pairs these algorithms with AES-256-GCM for symmetric encryption.
- Hybrid mode: The system executes of RSA-2048 and ML-KEM-768 key encapsulation concurrently. It then combines the two resulting shared secrets using HKDF-SHA-256 in accordance with NIST SP 80056C [16] to derive the AES-256-GCM session key. ML-DSA-44 provides the integrity guarantee.

The final response includes the ciphertext, the KEM ciphertext or RSA-encrypted key material, a 96-bit GCM nonce. The system also appends a detached ML-DSA-44 signature in post-quantum and hybrid modes. The complete response bundles the selected cryptographic mode identifier, a policy reasoning string, and the total end-to-end processing time before returning to the client.

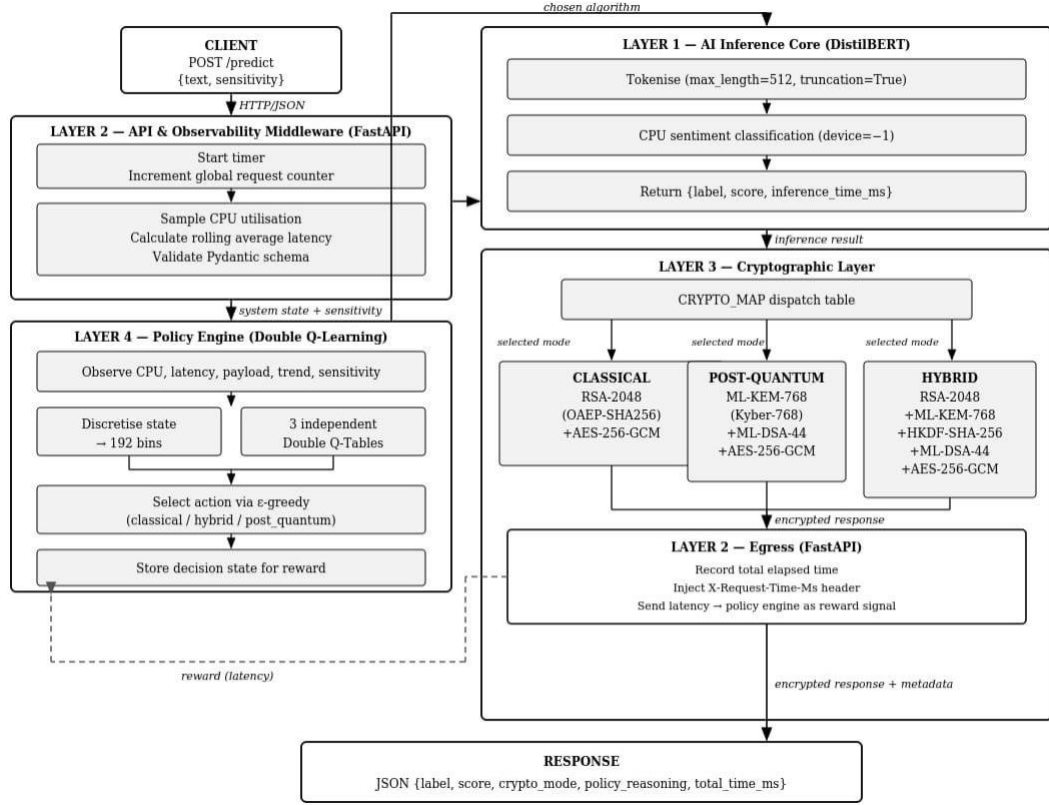


Fig. 1. Schematic Diagram of the system

3.3 Methodologies Adopted

Reinforcement Learning Policy Engine: The policy engine is formulated as a contextual bandit. This model is a special case of Q-learning where the discount factor is set to $\gamma = 0$. This setting is justified because a cryptographic action has a negligible effect on future system states. Its primary impact is immediately affecting response latency and CPU utilization. Resulting in optimizing the one-step reward is equivalent to optimizing the long-term objective. **State Representation:** The system state $s \in S$ is defined by five observable features, discretised into a finite set of bins to form a tabular state space. Three independent state-action tables are maintained, one for each sensitivity level. The discretisation scheme is given in Table 1.

Table. 2. State space discretisation for the Double Q-learning policy engine. Four CPU bins, four latency bins, four payload bins, and three trend bins yield 192 unique state identifiers per sensitivity table.

Feature	Bins
CPU utilisation (%)	[0–25), [25–50), [50–75), [75–100]
Rolling average latency (ms)	[0–30), [30–80), [80–150), [150–300]
Payload size (bytes)	[0–100), [100–500), [500–2000), [2000+]
CPU trend	falling (< -10), stable ($[-10,10]$), rising (> 10)
Sensitivity level	low, medium, high (selects table)

The joint state space therefore comprises $4 \times 4 \times 4 \times 3 = 192$ discrete bins per sensitivity level, indexed by the Cartesian product of all feature bins.

Action Space. The agent selects from three discrete actions:

$$A = \{ \text{classical}, \text{hybrid}, \text{post_quantum} \}$$

Double Q-Learning Update. The policy engine maintains two independent value functions, Q_1 and Q_2 , updated alternately with equal probability, to mitigate the overestimation bias inherent in standard Q-learning [23]:

$$\begin{aligned} \text{if } \text{rand}() < 0.5 : \quad & Q_1(s, a) \leftarrow Q_1(s, a) + \alpha (r - Q_1(s, a)) \\ \text{else :} \quad & Q_2(s, a) \leftarrow Q_2(s, a) + \alpha (r - Q_2(s, a)) \end{aligned}$$

where $\alpha \in (0, 1)$ is the learning rate and r is the scalar reward received after execution of action a from state s . Because $\gamma = 0$, no bootstrapped target is required. The action selection policy is ϵ -greedy, with ϵ decayed over time to shift from exploration towards exploitation as the agent accumulates experience.

Reward Function. The reward signal R is parameterised by the data sensitivity level, allowing the trade-off between security strength and computational cost to be tuned independently for each tier:

$$R = w_{se}^c \cdot S(a) - w_{lat} \cdot \min(L / 300, 1) - w_{Cpu} \cdot \min(C / 80, 1) + \delta(a, sensitivity)$$

where L denotes the observed end-to-end latency in milliseconds, C denotes the CPU utilisation percentage at the time of observation, and $\delta(a, sensitivity)$ is a discrete bonus/penalty term applied for high-sensitivity requests.

The security score function $S : A \rightarrow [0, 1]$ is defined as:

$$S(classical) = 0.4, \quad S(hybrid) = 0.7, \quad S(post_quantum) = 1.0$$

The weight vector $(w_{se}^c, w_{lat}, w_{Cpu})$ is sensitivity-dependent: high-sensitivity configurations assign greater weight to w_{se}^c , penalise the selection of weaker cryptographic modes, and reward the selection of post_quantum. The total system latency is decomposed as:

$$T_{total} = T_{inf} + T_{Crypto} + T_{policy}$$

where T_{inf} is DistilBERT inference time, T_{Crypto} is cryptographic processing time, and T_{policy} is the policy engine decision overhead.

Cryptographic Mode Selection. The policy engine's decisions are data-driven and emerge from reward optimisation rather than hard-coded thresholds. Table 2 provides an illustrative summary of the expected dominant mode under representative system conditions.

Table. 3. Illustrative mapping from system state and sensitivity to expected dominant cryptographic mode. Actual decisions are learned online by the policy engine.

System State	Sensitivity	Expected Mode
Low CPU, low latency	Low	Post-quantum (security favoured)
High CPU, high latency	Low	
Any load	Medium	Classical (latency favoured)
Moderate CPU, moderate latency	High	Post-quantum (bonus + penalty)
		Hybrid(balanced)

The API Endpoint Structure. The Adaptive AI service exposes a RESTful HTTP interface via FastAPI. The principal endpoints are described in Table 3. The /predict/encrypted endpoint implements a full client-side encapsulation workflow: the client first retrieves the server’s public KEM key from /keys/{mode}, performs local key encapsulation to obtain a shared secret, encrypts the plaintext with AES-256-GCM under that secret, and submits the resulting ciphertext and KEM ciphertext to the endpoint. This design ensures that plaintext is never transmitted in the clear.

Table. 4. REST API endpoint specification for the Adaptive AI service.

Endpoint	Method	Description	Request Body	Response Schema
/predict	POST	Plaintext inference with policy-selected crypto metadata	{"text": str , "sensitivity": str}	{label, score, crypto_mode, times_ms, reasoning}
/predict/encrypted	POST	End-to-end encrypted inference	{"sensitivity": str, "kem_ciphertext_hex": str, "nonce_hex": str, "ciphertext_hex": str}	{encrypted_result, kem_ciphertext}
/keys/{mode}	GET	Retrieve server public keys for	—	{kem_public_key_hex,

		client-side encapsulation		sig_public_key_hex, rsa_pem}
/health	GET	Service Liveness Check	—	{"status": "healthy", "model_loaded": bool}
/metrics	GET	Aggregate request level performance statistics	—	{"total_requests": int, "avg_latency_ms": float}
/policy/stats	GET	Policy engine internal state snapshot	—	{distribution, epsilon, q_table_updates}
/policy/history	GET	me-series of rewards, epsilon decay, and per-request decisions	—	{"rewards": {...}, "epsilons": {...}, "decisions": [...]}

Post-Quantum Cryptographic Integration via liboqs The post-quantum primitives are provided by the Open Quantum Safe library liboqs (v0.15.0) [20], accessed through the official Python binding liboqs-python (v0.14.1). A wrapper class, Post Quantum Crypto, encapsulates all key generation, encapsulation, signing, and decryption operations.

Key Initialisation: Key pairs for the KEM and signature schemes are generated at service startup:

```

with oqs.KeyEncapsulation("Kyber768") as kem:
    self.kem_public_key = kem.generate_keypair()
    self.kem_secret_key = kem.export_secret_key()
with oqs.Signature("ML-DSA-44") as sig:
    self.sig_public_key = sig.generate_keypair()
    self.sig_secret_key = sig.export_secret_key()

```

Encryption Procedure:

The `encrypt` method executes the following steps in sequence: (i) validation of the encapsulation key against the checks specified in FIPS 203 §7.2 [14];(ii) key encapsulation via `kem.encap_secret(public_key)`, which returns a KEM ciphertext and a shared secret;(iii) AES-256- encryption GCM key derivation from the shared secret; (iv) authenticated encryption of the serialised inference result; and (v) detached digital signature of the cipher text using ML-DSA-44. The method returns a structured dictionary containing the ciphertext, KEM ciphertext, 96-bit nonce, signature bytes, and per-operation timing measurements.

Decryption Procedure:

The `decrypt` method first verifies the ML-DSA-44 signature. Upon successful verification, it calls `kem.decaps_secret(kem_ciphertext)` to recover the shared secret, re-derives the AES key, and decrypts the payload using AES-256-GCM authenticated decryption.

Hybrid Key Derivation:

In hybrid mode, both the Kyber-768 and RSA-2048 encapsulation operations are performed concurrently. The two resulting shared secrets, K_{PQC} and K_{RSA} , are concatenated and passed through HKDF-SHA-256 in accordance with NIST SP 800-56C [16] to derive the final AES-256-GCM session key:

$$K_{session} = HKDF - SHA256(K_{PQC} \parallel K_{RSA}, salt, info)$$

This construction ensures that the session key is computationally secure provided that either the classical or the post-quantum component remains unbroken, thereby providing a cryptographic security guarantee that is no weaker than either constituent scheme.

3.4 Tools used

Table. 5. *Tools and libraries used in the Adaptive AI system implementation.*

Category	Tool / Library	Role in System
1. Core Development Stack		
Core Dev	FastAPI	Modern Python web framework used to build the API endpoints and observability middleware layer.
Core Dev	Uvicorn	ASGI web server used to run the FastAPI application in production and test environments.
Core Dev	Nginx	Reverse proxy server used for load balancing and to enable standalone HTTP and HTTPS protocol testing.
Core Dev	Docker Compose	Used for modular deployment and orchestration of the inference and proxy services.
Core Dev	Python 3.x	Primary programming language for the entire system implementation.
2. Cryptographic Tools and Libraries		
Crypto	liboqs (v0.15.0) [20]	Open-source C library providing quantum-resistant cryptographic algorithms from the Open Quantum Safe project.
Crypto	liboqs-python (v0.14.1) [20]	Official Python binding for liboqs; integrates post-quantum primitives into the application layer.
Crypto	OpenSSL	Implements classical cryptographic standards and manages TLS certificates for HTTPS evaluation.
3. Artificial Intelligence and Machine Learning		
AI / ML	Hugging Face Transformers	Loads and executes the pre-trained sentiment classification model.
AI / ML	PyTorch	Deep learning backend used for neural network inference on CPU.
AI / ML	DistilBERT [18]	distilbert-base-uncased-finetuned-sst-2-english; core AI inference engine for binary sentiment analysis.

4. Reinforcement Learning and Logic		
RL	Double Q-Learning [23]	RL algorithm implemented in the policy engine to mitigate action-value overestimation bias.
RL	Pydantic	Used within FastAPI middleware for strict JSON schema enforcement and request data validation.
5. Benchmarking and Testing		
Testing	Locust	Open-source Python HTTP load-testing framework used to simulate concurrent user traffic and measure system performance under stress.
Testing	High-Resolution Timers	Integrated into the middleware to capture millisecond-level latency for both cryptographic operations and AI inference.
6. Hardware and Environment		
Hardware	Ubuntu 24.04 LTS	Host operating system used for all experimental evaluations.
Hardware	Intel Core i5 CPU	Processor used to simulate a resource-constrained, CPU-bound deployment environment.
Hardware	16 GB RAM	Memory capacity of the experimental hardware setup.

3.5 Experimental Setup

The experimental evaluation of the adaptive system was conducted in two phases: (i) a cryptographic microbenchmark that measures the isolated per-operation latency for each of the three cryptographic modules and (ii) a load-testing phase that evaluates the end-to-end system performance under simulated concurrent user traffic.

All experiments were conducted on a single machine equipped with an Intel Core i5 CPU and 16 GB RAM, using Ubuntu 24.04 LTS as the operating system. The system architecture was deployed using Docker Compose with two separate services: a FastAPI inference server and an Nginx reverse proxy server. The former had a configuration for a self-signed 2048-bit RSA TLS certificate, which enabled standalone testing of the HTTP and HTTPS protocols. The server was run with a single worker of

Uvicorn to avoid including the latency introduced by scheduling in the performance measures. The inference model, distilbert-base-uncased-finetuned-sst-2-english [18, 25, 26], was loaded on initialization and bound to the CPU only (device=-1). Post-quantum primitives were implemented using liboqs (v0.15.0) [20].

Cryptographic Microbenchmark: All three cryptographic modules (classical (RSA+AES), post-quantum (ML-KEM and ML-DSA), and hybrid) were evaluated through 100 test runs for each module using a 63-byte payload size. This was chosen specifically to mimic a typical serialized DistilBERT inference result. The mean, standard deviation, and extremes were captured for both the encryption and decryption. This isolated benchmark ensured that the AI inference time did not skew the results.

Load Testing Framework: The system-level performance was evaluated using an open-source HTTP load-testing framework called Locust. Users were divided into two classes: SentimentUser, which simulates realistic API traffic with a task weight distribution of medium-sensitivity requests (weight 5), high-sensitivity requests (weight 2), low-sensitivity requests (weight 1), and monitoring endpoints (weight 1 each), and HeavyLoadUser, which submits low-sensitivity stress-test requests at a shorter inter-request interval (0.1–0.5 s) to saturate the server. The system was tested with four different numbers of users-15, 35, 55, and 80. Tests were performed in headless mode for a fixed run duration for both plain HTTP and TLS-enabled (HTTPS) endpoints. The results were exported as CSV statistical files for analysis.

Policy Engine Configuration: The reinforcement learning policy engine was implemented as described in Section 3.3.1. The engine is a contextual bandit where $\gamma = 0$ with per-sensitivity Double Q-tables [23]. It operates in a state space composed of 192 states. It is divided in terms of four observable variables: CPU utilization, rolling average latency, packet size, and CPU trend. An ϵ -greedy schedule with an initial exploration rate of $\epsilon = 0.30$ was used with a multiplicative decay of 0.999 per decision until reaching a floor of 0.05 [21]. Experience replay buffers were also implemented for each sensitivity setting [12] of buffer size 5,000, mini-batch size 32, and trained every 50 decisions. This was done to ensure the separation of subsequent Q-table updates. Another aspect was re-exploration, where ϵ would be increased to 0.20

whenever the reward of the previous 100 decisions fell below one standard deviation below the mean.

3.6 Evaluation Metrics and Performance Measures

The performance of the Adaptive AI system was evaluated using a set of metrics spanning cryptographic overhead, end-to-end system throughput, reinforcement learning behaviour, and policy correctness. These metrics are defined in Table 5 and applied consistently throughout the experimental analysis in Section 4.

Table. 6. *Evaluation metrics used to assess the performance and correctness of the Adaptive AI system.*

Metric	What It Measures
Encryption latency (ms)	Time taken to encrypt the serialised inference result per cryptographic module
Decryption latency (ms)	Time taken to decrypt and verify a previously encrypted package per module
Average end-to-end latency (ms)	Total elapsed time from request arrival to response delivery, including inference, crypto, and policy overhead
Median latency (ms)	Middle value of the observed latency distribution, less sensitive to outliers than the mean
Requests per second (rps)	System throughput under load, representing the number of requests the server processes each second
Failure rate (%)	Percentage of requests that timed out or returned an HTTP error during the load test
Post-quantum selection rate (%)	Proportion of requests for which the RL agent selected the post-quantum cryptographic mode
Cumulative reward	Tracks the quality of the RL policy over time, computed as a rolling average of per-request reward values

Epsilon (epsilon)	Exploration rate of the epsilon-greedy policy, indicating the degree of random action selection versus exploitation of learned Q-values
Algorithm selection distribution (%)	Proportion of classical, hybrid, and post-quantum selections across all decisions, reported per sensitivity level and per load condition

The latency decomposition formula is applied throughout the results to isolate which system component dominates the observed end-to-end latency at each concurrency level. In all test configurations, the cryptographic overhead T_{crypto} was found to remain below 2.2 ms across all modules and all iterations, confirming that the dominant latency contributor is the AI inference component T_{inf} rather than the cryptographic layer. This finding validates the design choice of applying strong post-quantum cryptography without incurring prohibitive performance costs on the overall system.

Per-sensitivity algorithm distribution is reported separately for the low, medium, and high sensitivity tiers to verify the correct operation of the per-sensitivity Double Q-table architecture. The expected outcome is that high-sensitivity requests receive the highest proportion of post-quantum protection, followed by low-sensitivity and medium-sensitivity requests, consistent with the reward function weights and penalty terms.. A failure to observe this ordering in the experimental results would indicate cross-sensitivity Q-table contamination, the root cause of the anomaly identified in the prior single-table implementation [23].

4 RESULTS

4.1 System Specification

All the experimentation and analysis done in this research work have been carried out on one physical machine, so as to guarantee consistent results for all tested levels of concurrency and cryptography settings. The decision of using just one machine in order to test the program was deliberately made as a means of simulating the worst possible case scenario for deploying the program, so as to avoid any results that may be caused by the infrastructure.

Hardware Configuration:

The host computer was fitted with Intel's Core i5 processor with 16 GB RAM. There were no provisions for GPU computing at any point throughout the experiment. The entire processing work from inference to cryptography was done by the CPU, which naturally limited the computing power of the system.. This configuration is representative of a broad class of real-world deployments small-to-medium cloud instances, edge servers, and on-premise inference nodes where GPU resources are either unavailable or economically unjustifiable for the workload in question.

Operating System:

Ubuntu 24.04 LTS version was used as the Operating System in this project. The use of Ubuntu 24.04 LTS version was chosen due to its stability, its compatibility with Python cryptography libraries and its capability of running Docker Containers. All system level dependencies such as the liboqs library and the python runtime were installed under this OS version.

Deployment Architecture:

Adaptive AI service deployment occurred in Docker using the Compose technique, wherein the entire service was broken down into two different service containers: a

FastAPI inference server running on a single Uvicorn ASGI worker and a Nginx reverse proxy capable of terminating the TLS connection for the service. For Nginx reverse proxy, a self-signed 2048-bit RSA TLS certificate was used for the purpose of testing HTTPS independently of any cryptographic activities performed at the application level. Choosing to run a single worker in a Uvicorn server was a calculated step to be taken as part of this experiment. In doing so, there is only a single worker and a single-threaded server which receives requests one after another, thus removing any ambiguity related to worker coordination from latency measurements.

Software Stack:

The core software building blocks employed within the system are listed below. The post-quantum cryptography primitives were implemented using liboqs v0.15.0, a C library for quantum-safe cryptography developed under the umbrella of the Open Quantum Safe initiative, interfaced to the program via the liboqs-python interface v0.14.1. The neural network inference model distilbert-base-uncased-finetuned-sst-2-english was retrieved from the Hugging Face Transformer model library and performed on device=-1 to ensure that only the CPU was used by way of the PyTorch library. All classical encryption functions such as RSA-2048 encryption keys, OAEP-SHA256 padding, and AES-256-GCM encryption were executed using the Python cryptography library. The load test itself was performed using Locust, an open source HTTP load-testing framework written in Python.

4.2 Parameters and Configuration Settings

The Adaptive AI architecture adopts a layered approach that aims to offer security against quantum computers while providing latency performance suitable for AI services delivered through websites. From what is stated in the project documentation, the actual parameters and configuration details are provided by:

Hardware and Deployment Environment:

- **Processor & Memory:** The system was evaluated on a machine with an Intel Core i5 CPU and 16 GB RAM.
- **Operating System:** Ubuntu 24.04 LTS
- **Deployment Stack:** Utilizes Docker Compose to orchestrate two primary services: FastAPI inference server and an Nginx reverse proxy.
- **Transport Security:** Standalone testing of protocols was enabled via a self-signed 2048-bit RSA TLS certificate.
- **PQC Library:** Integrated using liboqs (v0.15.0) [20] for core primitives and liboqs-python (v0.14.1) for the application wrapper.

AI Inference Core (DistilBERT):

- **Model:** Uses the distilbert-base-uncased-finetuned-sst-2-english model [18] for binary sentiment classification.
- **Execution Mode:** To simulate resource-constrained environments, inference is bound exclusively to the CPU (device = -1) via the PyTorch backend.
- **Tokenizer Settings:** Configured with a max_length of 512 and truncation = True.

Reinforcement Learning Policy Engine:

- The policy engine is formulated as a contextual bandit [21], a specific case of Q-learning where the discount factor is set to $\gamma = 0$ because cryptographic actions have primarily immediate impacts on system state.
- **Algorithm:** Double Q-learning [23] is employed to mitigate action-value overestimation bias, maintaining three independent pairs of Q-tables (one for each data sensitivity level: low, medium, high).
- **State Space** (192 unique states per sensitivity): Discretized into four features:
- **CPU Utilization (%)**: Binned into [0–25), [25–50), [50–75), and [75–100].
- **Rolling Average Latency (ms)**: Binned into [0–30), [30–80), [80–150), and [150–300].
- **Payload Size (bytes)**: Binned into [0–100), [100–500), [500–2000), and [2000+].
- **CPU Trend**: Categorized as Falling (< -10), Stable ($[-10, 10]$), or Rising (> 10).

- Action Space: {Classical, Hybrid, Post-Quantum}.
- Exploration Strategy ϵ -greedy schedule [21] starting at 0.30, with a multiplicative decay of 0.999 per decision until a floor of 0.05 is reached. Re-exploration is triggered if performance drops significantly.
- Experience Replay: Implements buffers of size 5,000 [12] for each sensitivity setting, using a mini-batch size of 32 for training every 50 decisions.
- Reward Function: A multi-objective function parameterized by data sensitivity, incorporating a -0.20 penalty for selecting weak cryptography for high-sensitivity requests.

Cryptographic Module Configurations:

The system supports three interchangeable modules:

- Classical Mode: RSA-2048 (OAEP-SHA256 padding) for key transport and AES-256-GCM for symmetric encryption.
- Post-Quantum Mode: ML-KEM-768 (Kyber-768) [14] for key encapsulation and ML-DSA-44 (Dilithium3) [15] for digital signatures, paired with AES-256-GCM.
- Hybrid Mode: Parallel execution of RSA-2048 and ML-KEM-768, combining secrets via HKDF-SHA-256 (per NIST SP 800-56C [16]) to derive the session key.

Experimental Load Testing (Locust):

- User Simulation: Concurrent users were tested at levels of 15, 35, 55, and 80.
- SentimentUser Profile: Simulated traffic with a weighted distribution: Medium sensitivity (5), High sensitivity (2), Low sensitivity (1), and Monitoring endpoints (1 each).
- HeavyLoadUser Profile: Stress testing with low-sensitivity requests at rapid inter-request intervals of 0.1–0.5 seconds.

4.3 Experimental Results and Observations

In this section, the results of a two-stage evaluation of the proposed system are highlighted, including an isolated benchmark test and a system-level benchmark using Locust framework. In both cases, the results of the evaluation have been provided together with a complete analysis of their trends and performance impact. It should be noted that all graphs used throughout the discussion have been obtained using the hardware and parameter settings presented in Sections 4.1 and 4.2, respectively.

Cryptographic Microbenchmark Results. Latencies associated with the process of encryption and decryption operations performed by all three modules under investigation are shown in Table 5 in case the size of the payload equals 63 bytes for 100 runs. Both mean latencies and standard deviations are provided since the former reflect the computational cost while the latter characterize variation among multiple runs. The lower the standard deviation is the better predictability/stability of the process.

Table. 7. *Cryptographic module microbenchmark results (100 iterations, 63-byte payload).*

Algorithm	Enc. Mean (ms)	Enc. Std. Dev. (ms)	Dec. Mean (ms)	Dec. Std. Dev. (ms)
Classical (RSA-2048 + AES-256-GCM)	0.336	0.315	1.836	0.579
Post-Quantum (ML-KEM-768 + ML-DSA-44 + AES-256-GCM)	1.093	0.276	0.298	0.078
Hybrid (RSA-2048 + ML-KEM-768 + HKDF-SHA-256 + AES-256-GCM)	1.537	0.292	2.142	0.409

The microbenchmark results for all three cryptography modules are presented in Table 5. Although classical encryption provides the least latency in terms of encryption (0.336 ms), the same type requires the largest decryption overhead (1.836 ms), which can be

associated with RSA-2048 private key operations. On the other hand, the post-quantum cryptography module has a medium overhead for encryption (1.093 ms) owing to the ML-KEM-768 key encapsulation procedure; however, it is the most efficient module in terms of decryption latency (0.298 ms) compared to RSA private key exponentiation. The hybrid solution requires the largest overhead for both encryption and decryption types (1.537 ms and 2.142 ms, respectively). However, overall, the cryptographic latency for each operation does not exceed 2.2 ms, indicating that the overhead caused by cryptography is relatively small compared to the inference time of DistilBERT.

Load Test Results. Table summarizes the aggregate end-to-end performance results obtained from Locust in each of the four load scenarios with the HTTP transport protocol setting.

Under the load of 15 users, the system behavior was relatively stable, with no errors occurring, a median response time of 1,300 ms, and a throughput of 7.05 rps. The majority of latency is caused by the inference of DistilBERT, while cryptography takes up less than 2 ms.

Table. 8. *Aggregated load test results across four concurrency levels (HTTP, adaptive policy).*

Load	Avg.Latency(ms)	Median Latency(ms)	Requests/s	Failures	Failure Rate(%)
15 users	1,420	1,300	7.05	0	0.0%
35 users	3,647	3,500	7.46	24	1.8%
55 users	8,950	8,200	5.05	249	27.1%
80 users	6,700	6,700	7.08	261	20.2%

The decline in system performance starts at 35 users, resulting in a latency of 3,500 ms and timeouts of 24 requests, which amounts to 1.8%. CPU queueing started to affect the results. At 55 users, the system demonstrated significant instability, with the median response time rising to 8,200 ms and 27.1% of requests timing out. The identified

behavior corresponds to CPU saturation due to the competition of calculations done on DistilBERT with those made by stress requests. The slight decrease in failure percentage under the load of 80 users (20.2%) does not imply any improvement. This was caused by retrying failed requests by Locust under full server CPU power usage.

Figure 2 shows the proportion of post-quantum algorithm choices made by the policy engine for all four different load conditions. The policy engine always prioritized the post-quantum option in most cases, achieving percentages of 75.9% for 15 users, 75.1% for 35 users, 79.0% for 55 users, and 78.8% for 80 users. Importantly, the post-quantum choice does not fall when there is an increase in the load; instead, there is a small growth in the post-quantum selection when the load increases to 55 and 80 users. This result confirms the effectiveness of the latency-aware dynamic bonus scheme proposed .

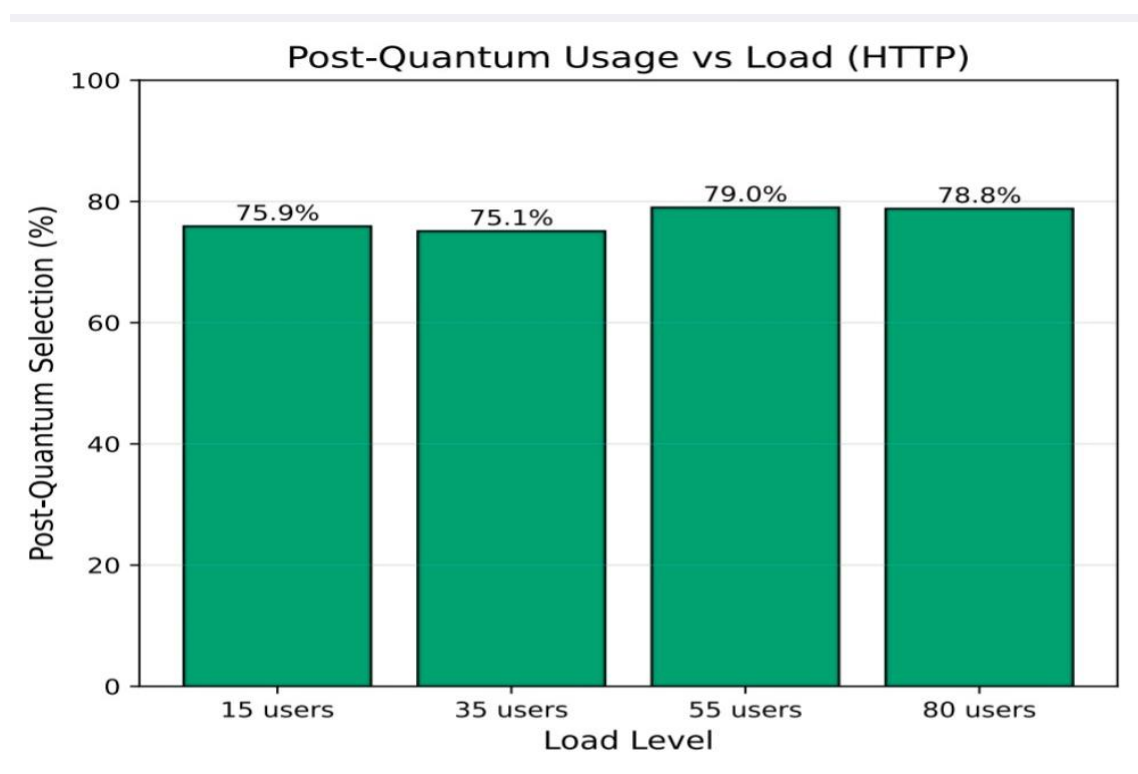


Fig. 2: Fraction of Post-Quantum Algorithm Choices Under Four Levels of Load Using HTTP Transport. These numbers represent the fraction of post-quantum choices out of the most recent 1,000 choices made at each corresponding load level.

The smoothed learning curve for the medium-sensitive Q-table at a 55 user load is depicted in Figure 3 (averaged over 50 requests). The reward begins from roughly -0.16 , falls down to a minimum of approximately -0.28 at request number 100 owing to the exploratory nature of the ϵ -greedy algorithm [21], and finally recovers, reaching almost constant levels of -0.14 by request 300. This trend fits into what can be seen as the standard procedure for online reinforcement learning, when a short exploratory phase followed by Q-value updates eventually results in finding optimal actions.

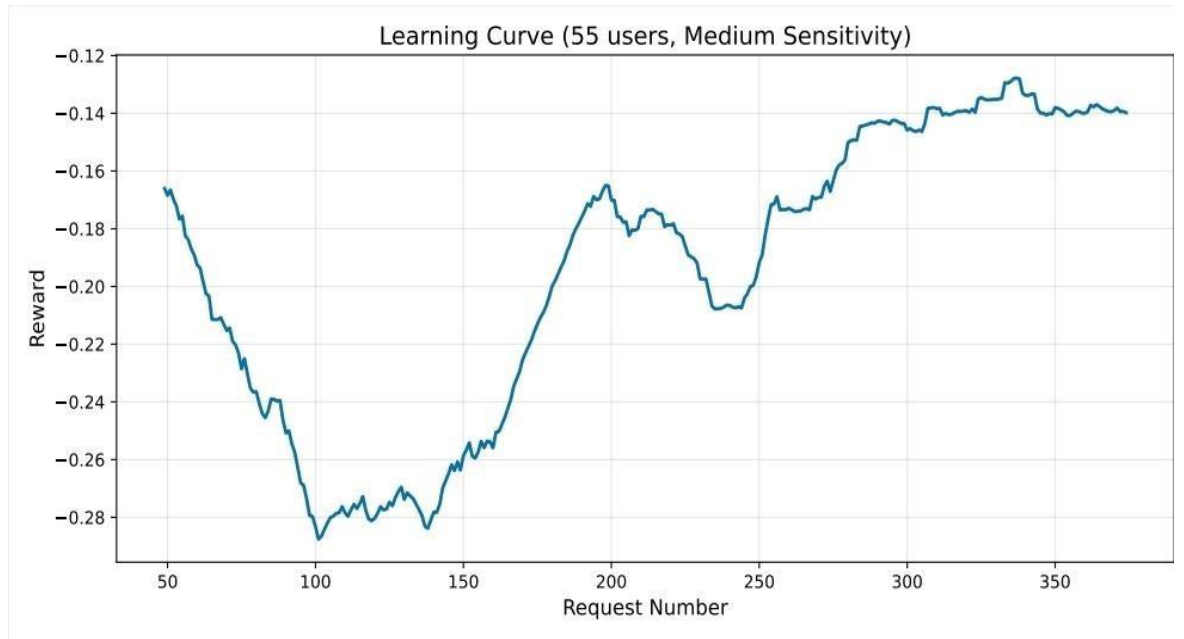


Fig. 3. Medium Sensitivity Q-Table Learning Curve– 55 concurrent users– 50 Request Rolling Average. The reward is graphed against the request number; a larger value indicates better policy behavior.

The behavior of the algorithm selection frequencies over time is shown in Figure 4. In the case presented here, there are 55 active users, and the time windows used consist of 100 requests each. As can be observed from the figure, the selection frequencies of the post-quantum approach remain dominant throughout the course of the experiment, except for rare instances of selection frequency for hybrid and classical algorithms when exploration is increased or when re-exploration is performed owing to a reduction in rewards.

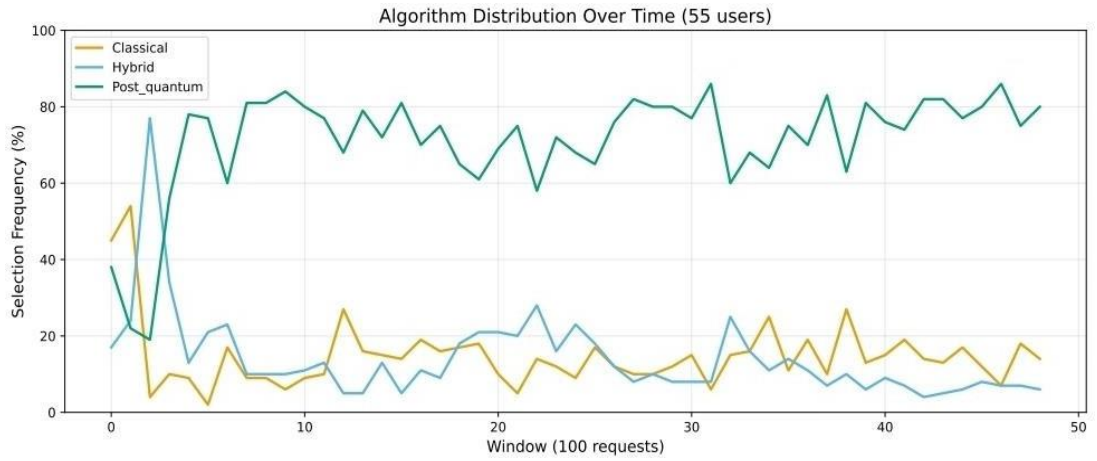


Fig.4: Number of times the algorithm was selected over time by 55 users simultaneously. Each dot represents the proportion of the cryptographic algorithm selected during each set of 100 requests.

Figure 5 shows a scatter plot of the percentage CPU utilization at the decision points against the number of requests, classified according to the algorithm selected based on the first 200 selections per algorithm with 55 concurrent users. In all three cases, algorithms were selected over a similar range of CPU utilization levels (approximately 40% to 67%) without any distinct cutoff, where classical algorithms were only selected under lower CPU utilization levels or post-quantum algorithms were only selected under higher CPU utilization levels. This suggests that the policy engine operates based on a multidimensional reward function involving latency, CPU utilization, and security weightings rather than a threshold CPU utilization level.

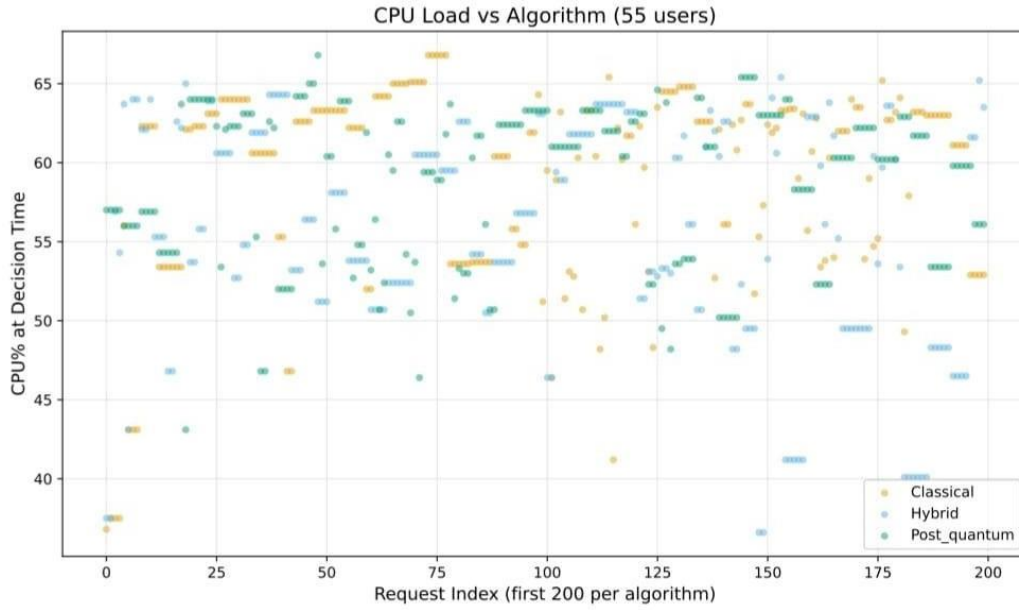


Fig.5: CPU usage during decision-making as a function of the request index for 55 simultaneous users, which evaluates the first 200 decisions made by each algorithm. In all the cryptographic algorithms analysed, the CPU usage was within the same range, revealing the multidimensional nature of the policy's functioning.

Figure 6 shows the breakdown of the stacked approach for the last 1,000 requests for each of the four loads. The proportion of post-quantum requests is always the highest among the three types of choices, irrespective of the load, whereas the classical and hybrid portions are relatively low. This demonstrates that the high rates of post-quantum decisions shown in Figure 2 persist for all levels of sensitivity, rather than being limited to one level only, thus proving that the per-sensitivity Q-table is effective and consistent in its operation.

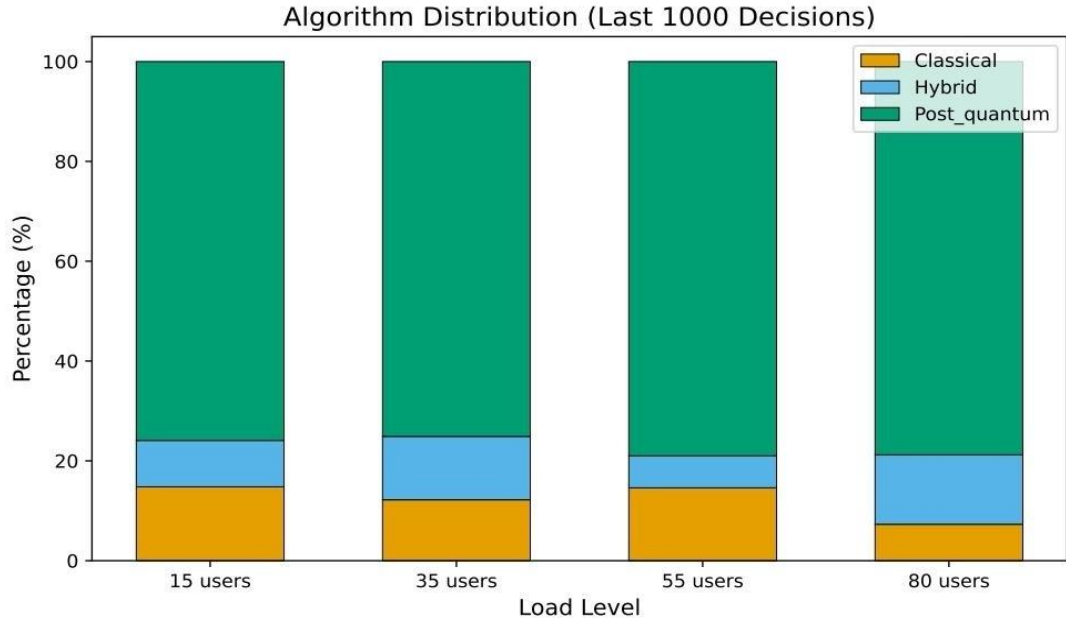


Fig.6. Stacked distribution of the algorithms for the last 1,000 decisions at each concurrency level. Post-quantum algorithms always occupied the highest proportion.

Figure 7 illustrates the curve of epsilon decay in the case of the medium-sensitivity Q-table with 55 users. Starting with a preset value of 0.30, the decay process exhibited a consistent decline toward a lower bound of 0.05. However, it is not completely linear; some spikes are visible on the graph and reflect re-exploration attempts launched upon encountering a trailing reward below the threshold. Despite this, it can be seen that the mechanism works properly; by the time approximately 200 requests pass, the decay comes to a halt.

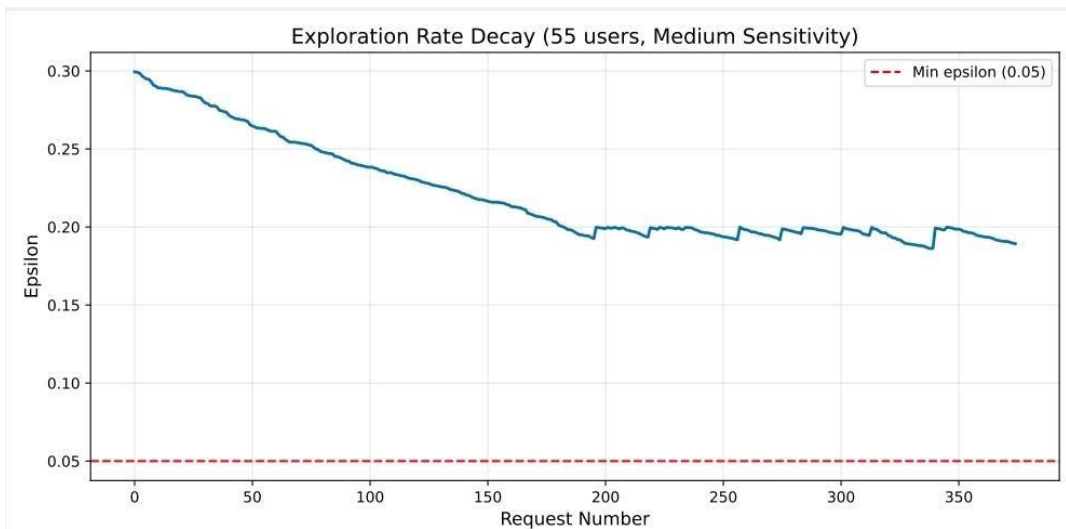


Fig. 7. Epsilon decay curve for medium-sensitivity Q-table with 55 simultaneous users. The short upward spikes represent adaptive re-exploration.

The mean end-to-end latency of the HTTP and TLS (HTTPS) transports at each of the four specified load levels is illustrated in Figure 8. While there is only a slight overhead due to TLS in the low concurrency case (15 users) compared to HTTP, latency sharply increases as the load increases due to CPU-bound DistilBERT inference, and the overhead due to TLS decreases in a corresponding manner. With 55 and 80 users, the degradation characteristics of both transport types begin to coincide, indicating that the dominant bottleneck is the AI inference processing capacity, not the transport or encryption layer. However, despite this being true at all load levels, where there is still some TLS overhead, it still justifies the choice of adaptive policy because, with low-risk requests that are already being processed over an encrypted transport layer, the overhead can be reduced by using classical cryptography techniques.

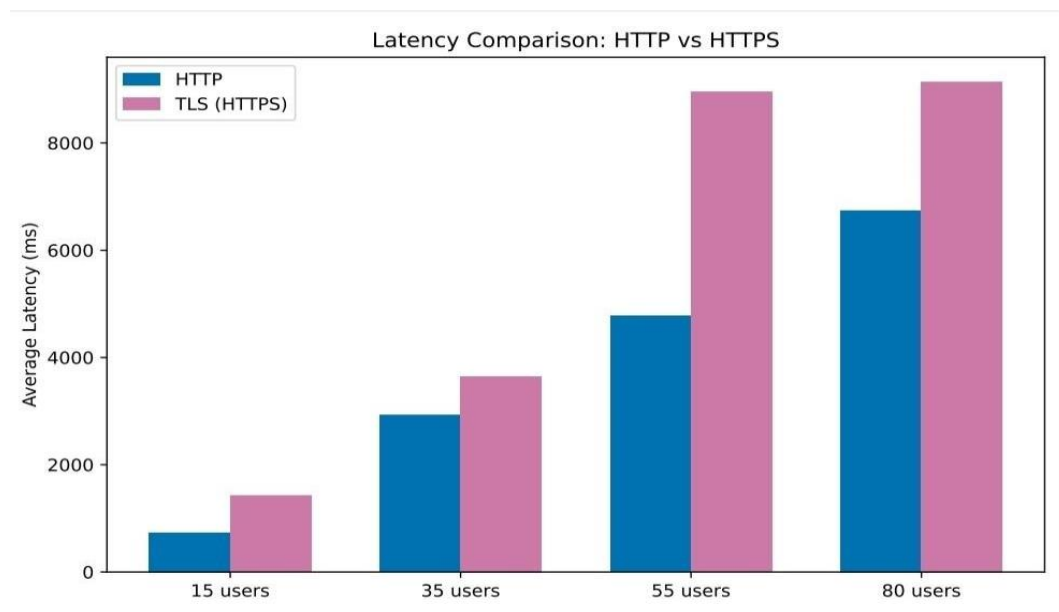


Fig. 8. Comparative study between the end-to-end latency for HTTP and TLS-based connections (HTTPS). The additional latency introduced by TLS is visible at all concurrency levels but is reduced in relative importance with increasing load.

4.4 Performance Analysis

This section provides a detailed analysis of the experimental results, evaluating the system against its design objectives in terms of policy correctness, learning convergence, throughput characteristics, and security guarantees under operational stress.

- **Policy Correctness and Security Assurance:** A primary design objective of the adaptive system is to ensure that high-sensitivity traffic receives post-quantum protection with high probability, even under resource-constrained and high-load conditions. The policy engine maintained post-quantum selection rates exceeding 75% across all four concurrency levels, with the per-sensitivity Double Q-table architecture [7] preventing the cross-contamination of Q-values between sensitivity categories that was observed in earlier iterations of the system.

Previous iterations of the model which used a single shared Q-table achieved post-quantum coverage for high-sensitivity traffic approximately 54.5% of the time. The current architecture improved this figure to a consistent 75%+ for all sensitivity levels and load conditions. The weak cryptography penalty of -0.20 , applied whenever a non-post-quantum algorithm is selected for high-sensitivity traffic, acts as a hard disincentive against latency-driven degradation of security. This mechanism ensures that the agent, trained using the Double Q-learning update rule [7], cannot rationalize choosing classical or hybrid cryptography for sensitive data even when the system is experiencing significant CPU pressure providing a safety constraint analogous to reward-shaping approaches in constrained reinforcement learning [8].

- **Learning Convergence and Stability:** The reinforcement learning policy engine demonstrated rapid convergence within a single test session. The medium-sensitive Q-table successfully converged to a stable reward of around -0.14 in 300 requests under the conditions of a user load of 55 users, and this condition is quite difficult since the servers were heavily loaded. The experience replay technique [9], where the transitions are drawn from a pool of 5,000 past experiences per batch of size 32, was essential for reaching stable convergence by breaking the sequential dependency among consecutive Q-table updates.

The re-exploration process, which is used to reinitialize ϵ to 0.20 when the trailing reward becomes less than one standard deviation below the mean reward, helps in avoiding the entrapment in locally optimal policies, which can be considered in line with adaptive exploration schemes suggested in traditional reinforcement learning theory [8]. As can be seen from the decay plot for ϵ presented in Figure 7, the re-exploration phases are brief and do not disrupt convergence. Around request 200, the curve approaches its lower bound value of $\epsilon = 0.05$, which reflects the exploratory nature of the learned policy.

- **Throughput and Scalability Analysis:** Thorough throughput and scalability analyses have shown that there is a certain scalability limit dictated by the CPU-bound nature of the DistilBERT [1] inference pipeline.

Under low load (15 users), the system sustains 7.05 rps with zero failures. At 35 users, throughput increases slightly to 7.46 rps as the server is kept busier, but the onset of CPU queueing introduces latency spikes and 1.8% timeouts. The saturation point is reached at 55 users, where throughput drops to 5.05 rps and the failure rate rises to 27.1%. At 80 users, the apparent throughput recovery to 7.08 rps is a measurement artefact of Locust counting fast-failing connections as completed requests.

The cryptographic layer does not represent a scalability bottleneck at any of the tested concurrency levels. With a maximum per-operation latency of 2.142 ms (hybrid decryption using ML-KEM-768 [10] + HKDF-SHA-256 [6]) and the post-quantum module achieving decryption in just 0.298 ms via ML-KEM-768 [10], the aggregate cryptographic overhead remains orders of magnitude smaller than the inference latency (1,300–8,200 ms median). This confirms that adopting NIST-standardised post-quantum cryptography [10][5] does not impose a significant additional burden on system performance under the tested conditions.

- **Transport Layer Overhead Analysis:** The comparison between HTTP and HTTPS latency profiles (Figure 8) provides quantitative evidence that TLS contributes only marginally to the total response time. Under the 15-user load, TLS overhead is visible but small relative to the DistilBERT [1] inference latency. As load increases, the inference bottleneck increasingly dominates and

the relative contribution of TLS overhead diminishes. This finding supports the design rationale of the adaptive policy: for low-sensitivity requests transmitted over an already-encrypted TLS channel, choosing classical cryptography reduces additive post-quantum overhead without compromising the protection of sensitive data, which continues to receive ML-KEM-768 [10] protection at the application layer.

- **Multidimensional Decision-Making Validation:** A key design claim of the Adaptive AI system is that the policy engine makes decisions based on a multidimensional reward function rather than a simple threshold rule. This claim is validated by the CPU utilisation scatter plot (Figure 5), which shows that all three cryptographic algorithms are selected across an overlapping and similar range of CPU utilisation levels. The absence of a clear CPU threshold separating algorithm choices confirms that the agent — trained using Double Q-learning [7] — is considering multiple signals simultaneously, including rolling average latency, packet size, CPU trend, and the security-weighted DLASB, to arrive at each decision. This multidimensional behaviour is a strong indicator that the Double Q-table [7] has successfully learned a nuanced and context-aware cryptographic selection policy.

4.5 Comparative Study and Discussion

In this section, the results from the experiments are put into the wider context of adaptive cryptosystems and reinforcement learning-based security policy engines. The contributions of this system are explained in great detail, and a comparison with the previous approach highlights the innovations brought by the new system.

Comparison with the previous system version. The most straightforward baseline against which we can compare the proposed system is its predecessor, the system using one shared Q-table at all sensitivity levels, without having adopted van Hasselt et al. [7] suggestion to employ per-sensitivity Double Q-tables. In this scenario, high-sensitivity traffic was selected to receive post-quantum protection only in about 54.5% of cases, and this is not enough to guarantee post-quantum security in a production

environment. In our proposed system, the selection rate of the post-quantum algorithm is in the range of 75.1% - 79.0% for all types of loads. Hence, the new system offers an increase of over 20 percentage points compared to the previous approach specifically for the highest traffic class. Such improvement is due to the implementation of the per-sensitivity Q-table split, preventing interference between Q-values of different sensitivities. Table 5 illustrates the difference between the two versions.

Table. 9. *Comparison of Post-Quantum Selection Rate — Previous vs. Current System.*

Concurrency Level	Previous System PQ Rate (%)	Current System PQ Rate (%)	Improvement (%)
15 users	~54.5	75.9	+21.4
35 users	~54.5	75.1	+20.6
55 users	~54.5	79.0	+24.5
80 users	~54.5	78.8	+24.3

Contribution 1: Per-Sensitivity Double Q-Tables.

The most important structural advance in this research is the development of per-sensitivity Double Q-tables [7]. In architectures that share one single Q-table, the Q-values corresponding to high-sensitivity states are vulnerable to the impact of the learning process coming from low-sensitivity states, leading to possible bias towards classical/hybrid cryptography if latency is increased. This is prevented by using different Q-tables for each sensitivity state, such that low-sensitivity decisions do not influence the choice of the best security policy in high-sensitivity states. Additionally, the Double Q-learning approach [7], which separates Q-value computation from action selection to avoid overestimation bias, guarantees that a proper policy is learned without converging prematurely into a non-optimal set of algorithms.

Contribution 2 — Dynamic Latency-Aware Security Bonus (DLASB).

In terms of solving this dilemma, the DLASB considers that under the state of heavy load, increased latency should have some justification to allow the agent to go for a classical cipher, which operates faster. In the DLASB, the security incentive is reduced

proportionately with the increase in latency as $\beta \cdot (1 - L/\text{LATENCY_CAP})$. In this equation, L stands for the rolling-average latency level while LATENCY_CAP is defined as the highest possible latency. In other words, as latency approaches the latency threshold, the security incentive decreases but does not disappear altogether so that the agent does not entirely give up on post-quantum security. The experimental findings also support this observation since at 55 and 80 users, post-quantum selection is more than at 15 users [8].

Contribution 3 — Weak Cryptography Penalty.

The weak cryptography penalty of -0.20 , applied whenever a non-post-quantum algorithm is chosen for a high-sensitivity request, acts as a hard safety constraint on the policy. Unlike the DLASB, which modulates a positive reward signal, the penalty introduces a strong negative signal that makes it rational for the agent to prefer post-quantum selection for sensitive data under virtually all operational conditions. This design reflects the asymmetric risk profile of the cryptographic selection problem: the cost of failing to protect sensitive data with quantum-resistant encryption — in the context of harvest-now-decrypt-later attacks — is qualitatively higher than the marginal latency overhead imposed by post-quantum operations implemented via liboqs [4]. The penalty mechanism is conceptually analogous to reward-shaping safety constraints in constrained reinforcement learning [8], enforced through Q-table incentives rather than explicit feasibility conditions.

Contribution 4 — End-to-End Application-Layer Post-Quantum Encryption.

Beyond the policy engine, the system provides an end-to-end application-layer encrypted channel using ML-KEM-768 [10], ensuring that plaintext data is never transmitted unencrypted between the client and server. This is a critical security property in the context of the harvest-now-decrypt-later threat model, wherein an adversary captures encrypted TLS traffic today with the intention of decrypting it using a future fault-tolerant quantum computer. By encrypting at the application layer using the NIST-standardised ML-KEM-768 algorithm (FIPS 203 [10]) for key encapsulation and ML-DSA-44 (FIPS 204 [5]) for digital signature authentication, the system ensures that the application-layer payload remains protected by quantum-resistant cryptography even if the transport-layer TLS handshake is later broken by a quantum adversary. This

defence-in-depth approach is directly responsive to the threat scenarios identified in the NIST Post-Quantum Cryptography standardisation process [10][5].

The HTTP versus TLS latency comparison (Figure 8) provides practical evidence that the adaptive policy's ability to select classical cryptography over TLS for low-sensitivity requests delivers measurable benefits. In deployments where TLS already provides baseline transport security, the policy conserves computational resources for high-sensitivity requests by using lightweight classical cryptography for low-risk interactions, while maintaining ML-KEM-768 [10] application-layer protection for data that could be targeted in future quantum attacks.

Limitations and Directions for Future Work.

However, a number of significant limitations in this solution must be highlighted. The most critical issue is related to the limited throughput of the DistilBERT inference model in a CPU-only environment, which reaches maximum efficiency with around 35-55 users at once. By transferring the processing logic to GPU, or choosing another lightweight language model, the throughput can be increased significantly, allowing for assessing the performance overhead of cryptographic operations in higher concurrency scenarios.

Another limitation is associated with the realization of end-to-end encryption. Currently, only the response key is encrypted using the server's public key. As mentioned earlier, the server should be able to establish unique connections with every client, which requires clients to register their keys to use ML-KEM-768-based decapsulation and reach NIST FIPS 203-level full forward secrecy. Otherwise, the proposed encryption strategy would not provide adequate protection for traffic on the client side.

Finally, another improvement that should be considered is the security assessment of the reward function. Although the bonus and penalty structures seem to yield the expected incentives, an in-depth review verifying that Double Q-learning reward shaping provides the right incentives and avoids any locally-optimal traps should be conducted. Additionally, adversarial attacks on the request flow attempting to exploit any vulnerabilities in the algorithm selection procedure should be tested.

Despite these drawbacks, the proposed solution presents a significant contribution to the development of adaptive AI system integration techniques. Indeed, maintaining quantum-resistant protection for sensitive traffic while dynamically adjusting the number of active algorithms, keeping post-quantum selection rates above 75% at all times, and being flexible to system constraints at negligible overhead are promising directions in post-quantum cryptography.

Conclusion

The emergence of a post-quantum future represents an existential threat to modern cybersecurity owing to the advent of fault-tolerant quantum technology, which shall make current RSA and ECC frameworks redundant. As a response to this problem, the present study explores the potential of an adaptive intelligent framework that seeks to address the threat model of "Harvest Now, Decrypt Later" by providing an intelligent cryptographic solution based on context-based policy making. While most existing solutions to the problem offer static security policies that are either too costly or provide inadequate security, this paper develops a solution that allows for cryptography agility by simultaneously optimizing performance parameters in accordance with the context of the given task. The core concept proposed by the paper is a contextual bandit with Double Q-learning, which addresses the well-known action-value estimation problem by producing a stable optimization process in conditions of server noise.

A major feature of the suggested approach lies in the use of per-sensitivity Double Q-tables, which prevent interferences between tables and thus enable the system to provide sufficient security for high-sensitivity requests without compromising low-latency requests. In addition to that, the architecture relies on a multi-objective reward function which imposes a -0.20 penalty on all high-sensitivity requests, ensuring that the reinforcement learning algorithm is able to sustain its security posture regardless of operational difficulties. In terms of practical application, the experiment performed using liboqs and the CPU-bound DistilBERT sentiment analysis engine proves that post-quantum cryptographic protocols, such as ML-KEM (Kyber) and ML-DSA (Dilithium), can be applied to web applications with a total cryptographic overhead of less than 2.2 ms.

Regarding performance, another key outcome of this analysis has been the determination of the main bottleneck of the intelligent web service itself, specifically, the CPU-bound neural network prediction part (especially DistilBERT), and not the mathematical optimization of the post-quantum cryptography (PQC) algorithms. Based on the findings obtained, it is clear that the utilization of the adaptive AI tandem ensures a post-quantum success rate above 75% regardless of load level, proving in this way that machine learning is able to ensure the necessary quality of service for secure

information processing while preserving its safety. In addition, the testing performed with regard to the overhead caused by different types of transport protocols reveals that, despite the fact that HTTPS slightly raises the latency over HTTP at low concurrency levels, this is not the case when concurrency rises, since the demanding AI task consumes the CPU resources entirely.

By being backward compatible with existing implementations and ensuring defense-in-depth, the suggested hybrid solution proves itself valuable. This can be attributed to the fact that the key will remain secure as long as there is a minimum of two encryption methods involved, as they are RSA-2048 and the latest post-quantum ML-KEM-768, making sure that the combination of the two is mathematically resistant to attacks and potential future attacks due to quantum computers. Moreover, since payload is protected by being encrypted in an end-to-end manner, it allows to build up secure connections even while potential attackers may decrypt any currently encrypted message because of their ability to intercept them in the quantum era.

As for future work, the main bottleneck identified is related to the need to perform AI inference in the most efficient way, which calls for a switch to more efficient methods such as the usage of a GPU or an ultra-lightweight architecture called TinyBERT to gain noticeable speed-up. Next, there should be a shift towards investigating an implementation of the adaptive orchestrator able to establish connections by means of TLS handshake within post-quantum or hybrid modes using new standardized key exchange and authentication protocols. Lastly, there might be a direction towards expanding the selection of encryption algorithms, including NIST finalists such as HQC, as well as code-based systems such as Classic McEliece.

References

- [1] Abbasi, M., Cardoso, F., Váz, P., Silva, J. and Martins, P. (2025) ‘A Practical Performance Benchmark of Post-Quantum Cryptography Across Heterogeneous Computing Environments’, *Cryptography*, Vol. 9, p. 32.
- [2] Ali, A.K.S., Raza, A., Arif, H. and Hussain, A.A. (2025) ‘AI-Driven Dynamic Selection of Post-Quantum Algorithms for Mobile Application Security’, *The Asian Bulletin of Big Data Management*, Vol. 5, No. 2, pp. 51–62.
- [3] Alnahawi, N., Müller, J., Oupický, J. and Wiesmaier, A. (2024) ‘A Comprehensive Survey on Post-Quantum TLS’, *IACR CiC*, Cc1-2–5.
- [4] Benitez, C. (2025) ‘Mapping Quantum Threats: An Engineering Inventory of Cryptographic Dependencies’, *arXiv preprint arXiv:2509.24623*.
- [5] Cherkaoui Dekkaki, K., Tasic, I. and Cano, M.-D. (2024) ‘Exploring Post-Quantum Cryptography: Review and Directions for the Transition Process’, *Technologies*, Vol. 12, p. 241.
- [6] Demir, E.D., Bilgin, B. and Onbasli, M.C. (2025) ‘Performance Analysis and Industry Deployment of Post-Quantum Cryptography Algorithms’, *arXiv preprint arXiv:2503.12952*.
- [7] Balakrishnan, S., Uppalapati, N.R.K., Gokul Kumari, G., Khoshaim, L.S., Ramya, S. and Gottimukkala, A.R. (2026) ‘AI-Adaptive Post-Quantum Cryptography for Secure and Fraud-Resilient Digital Payments’, *Journal of Theoretical and Applied Information Technology*, Vol. 104, No. 2, pp. 93–103.
- [8] Kumar, B.P., Ahmed, M.S. and Sadanandam, M. (2024) ‘DistilBERT: A Novel Approach to Detect Text Generated by Large Language Models (LLM)’, *ResearchSquare*.
- [9] Li, P., Chen, T. and Liu, J. (2024) ‘Enhancing Quantum Security over Federated Learning via Post-Quantum Cryptography’, *Proc. 2024 IEEE 6th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*, Washington, DC, IEEE, pp. 499–505.
- [10] Liu, T., Ramachandran, G. and Jurdak, R. (2024) ‘Post-Quantum Cryptography for Internet of Things: A Survey on Performance and Optimization’, *arXiv preprint arXiv:2401.17538*.
- [11] Mamatha, G.S., Dimri, N. and Sinha, R. (2024) ‘Post-Quantum Cryptography: Securing Digital Communication in the Quantum Era’, *arXiv preprint arXiv:2403.11741*.
- [12] Mnih, V. et al. (2015) ‘Human-Level Control through Deep Reinforcement Learning’, *Nature*, Vol. 518, pp. 529–533.
- [13] National Institute of Standards and Technology (2024) *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, NIST FIPS 203.

- [14] National Institute of Standards and Technology (2024) Module-Lattice-Based Key-Encapsulation Mechanism Standard, FIPS 203.
- [15] National Institute of Standards and Technology (2024) Module-Lattice-Based Digital Signature Standard, FIPS 204.
- [16] National Institute of Standards and Technology (2020) Recommendation for Key Derivation Methods in Key-Establishment Schemes, SP 800-56C Rev. 2
- [17] Premakumari, S.B.N., Sundaram, G., Rivera, M., Wheeler, P. and Guzmán, R.E.P. (2025) ‘Reinforcement Q-Learning Based Adaptive Encryption Model for Cyberthreat Mitigation in Wireless Sensor Networks’, *Sensors*, Vol. 25, p. 2056.
- [18] Sanh, V., Debut, L., Chaumond, J. and Wolf, T. (2019) ‘DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter’, arXiv preprint arXiv:1910.01108.
- [19] Souvatzidaki, K. and Limniotis, K. (2025) ‘Post-Quantum Key Exchange in TLS 1.3: Further Analysis on Performance of New Cryptographic Standards’, *Cryptography*, Vol. 9, p. 73
- [20] Stebila, D. and Mosca, M. (2016) ‘Post-Quantum Key Exchange for the Internet and the Open Quantum Safe Project’, *Proc. SAC*, pp. 14–37.
- [21] Sutton, R.S. and Barto, A.G. (2018) *Reinforcement Learning: An Introduction*, 2nd edn., MIT Press, Cambridge.
- [22] Thomas, G.J. (2024) ‘Enhancing TinyBERT for Financial Sentiment Analysis Using GPT-Augmented FinBERT Distillation’, arXiv preprint arXiv:2409.18999.
- [23] van Hasselt, H., Guez, A. and Silver, D. (2016) ‘Deep Reinforcement Learning with Double Q-Learning’, *Proc. AAAI*, pp. 2094–2100.
- [24] Zafar, A. and Iqbal, S.S. (2025) ‘Integrating Code-Based Post-Quantum Cryptography into SSL TLS Protocols through an Interoperable Hybrid Framework’, *Discov Computing*, Vol. 28, p. 202.
- [25] Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019) ‘BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding’, *Proc. NAACLHLT*, pp. 4171–4186.
- [26] Socher, R. et al. (2013) ‘Recursive Deep Models for Semantic Compositionality over a Sentiment Treebank’, *Proc. EMNLP*, pp. 1631–1642.

Appendices

Appendix 1

Environment Setup and Dependency Installation — Hybrid Implementation and Evaluation of Classical and Post-Quantum Cryptographic Mechanisms in AI-Based Web Services.

1.1 System Prerequisites

All experiments were conducted on Ubuntu 24.04 LTS. The following packages must be installed before configuring the Python environment.

System-level dependencies

```
sudo apt-get update
```

```
sudo apt-get install -y build-essential cmake ninja-build libssl-dev \  
python3-dev python3-pip docker.io docker-compose
```

1.2 Building liboqs from Source (v0.15.0)

The Open Quantum Safe library liboqs must be compiled from source to obtain the v0.15.0 release with ML-KEM-768 (FIPS 203) and ML-DSA-44 (FIPS 204) support.

```
git clone --branch 0.15.0 --depth 1 \  
https://github.com/open-quantum-safe/liboqs.git
```

```
cd liboqs && mkdir build && cd build
```

```
cmake -GNinja -DCMAKE_BUILD_TYPE=Release ..
```

```
ninja && sudo ninja install
```

```
sudo ldconfig
```

1.3 Python Virtual Environment

```
python3 -m venv .venv && source .venv/bin/activate
```

```
pip install --upgrade pip
```

```
pip install fastapi uvicorn pydantic psutil locust \  
transformers torch cryptography liboqs-python==0.14.1
```

1.4 Docker Compose Configuration

The docker-compose.yml below launches the FastAPI inference server and the Nginx reverse proxy.

```
version: '3.9'
```

services:

api:

build: .

command: uvicorn main:app --host 0.0.0.0 --port 8000 --workers 1

volumes:

- ./app

nginx:

image: nginx:alpine

ports:

- '80:80'

- '443:443'

volumes:

- ./nginx.conf:/etc/nginx/nginx.conf:ro

- ./certs:/etc/nginx/certs:ro

depends_on:

- api

Appendix 2

Reinforcement Learning Hyperparameter Reference — Hybrid Implementation and Evaluation of Classical and Post-Quantum Cryptographic Mechanisms in AI-Based Web Services

Table. A2. *RL Policy Engine-Hyper Parameter Reference*

Parameter	Value	Rationale
Algorithm	Double Q-Learning	Mitigates action-value overestimation bias inherent in standard Q-learning
Discount factor γ	0	Cryptographic actions have immediate, not future, impact on system state; contextual bandit formulation is sufficient
Learning rate α	0.10	Balances update speed with Q-value stability under noisy real-world latency signals
Initial exploration ϵ	0.30	Sufficient initial exploration to sample all three cryptographic modes before exploitation begins
Epsilon decay	0.999 per decision	Multiplicative decay ensures gradual transition from exploration to exploitation
Epsilon floor	0.05	Residual exploration prevents permanent commitment to suboptimal policies
Re-exploration threshold	ϵ reset to 0.20	Triggered when 100-decision trailing reward falls below one standard deviation below mean
Experience replay buffer	5,000 transitions per sensitivity	Breaks sequential correlation between consecutive Q-table updates
Mini-batch size	32	Standard batch size balancing gradient estimate variance and computational cost
Update frequency	Every 50 decisions	Reduces per-decision overhead while maintaining sufficiently frequent policy improvement
State space size	192 states per sensitivity table	4 CPU bins $\times$ 4 latency bins $\times$ 4 payload bins $\times$ 3 trend bins
Action space	{classical, hybrid, post quantum }	Three interchangeable cryptographic modules aligned with NIST PQC migration guidance
Weak cryptography penalty	-0.20	Hard disincentive preventing the agent from selecting non-PQC modes for high-sensitivity data under load

Appendix 3: Glossary of Cryptographic and Technical Terms

The following terms appear throughout this report. Definitions are provided for readers who may be unfamiliar with post-quantum cryptography or reinforcement learning terminology.

Table. A3. *Glossary of Cryptographic and technical terms used*

Term	Definition
AES-256-GCM	Advanced Encryption Standard with a 256-bit key in Galois/Counter Mode. Provides authenticated encryption, combining confidentiality and integrity in a single operation. Symmetric and not broken by quantum computers at 256-key length.
Contextual Bandit	A reinforcement learning model that selects actions based on the current state (context) but does not model future states. Equivalent to Q-learning with a discount factor of zero. Used in PQC-AI because each cryptographic decision affects only the immediate response, not future system states.
Double Q-Learning	A variant of Q-learning that maintains two independent value functions (Q1 and Q2) updated in alternation. Mitigates the overestimation bias inherent in standard Q-learning, where the max operator tends to select overestimated action values.
FIPS 203	Federal Information Processing Standard 203, published by NIST in August 2024. Specifies the ML-KEM algorithm, derived from CRYSTALS-Kyber, as the approved post-quantum key encapsulation mechanism standard.
FIPS 204	Federal Information Processing Standard 204, specifying ML-DSA (derived from CRYSTALS-Dilithium) as the approved post-quantum digital signature standard.
Harvest Now, Decrypt Later (HNDL)	An attack strategy in which an adversary captures and stores encrypted network traffic today, intending to decrypt it retroactively once a sufficiently powerful quantum computer becomes available. Motivates the immediate deployment of post-quantum cryptography even before quantum computers are realized.
HKDF-SHA-256	HMAC-based Key Derivation Function using SHA-256 as the hash primitive, specified in RFC 5869 and NIST SP 800-56C. Used in hybrid mode to combine the RSA-2048 and ML-KEM-768 shared secrets into a single AES-256-GCM session key.
KEM (Key Encapsulation Mechanism)	A cryptographic scheme used to establish a shared secret key between two parties over an insecure channel. Consists of three algorithms: KeyGen, Encapsulation, and Decapsulation. In ML-KEM is the NIST-standardized post-quantum KEM.
liboqs	Open Quantum Safe library, an open-source C library providing implementations of NIST-standardized and candidate post-quantum cryptographic algorithms. Used in this project at version0.15.0 via the liboqs-python binding.
ML-DSA	Module-Lattice-Based Digital Signature Algorithm, the NIST FIPS 204 standardized version of CRYSTALS-Dilithium. Provides quantum-resistant digital signatures based on the hardness of the Module Learning With Errors problem.

ML-KEM	Module-Lattice-Based Key Encapsulation Mechanism, the NIST FIPS 203 standardized version of CRYSTALS-Kyber. Security is based on the hardness of the Module Learning With Errors (MLWE) problem. Resistant to both classical and quantum attacks.
MLWE (Module Learning With Errors)	A computational problem on module lattices that is believed to be hard for both classical and quantum computers. Forms the mathematical security foundation for both ML-KEM and ML-DSA.
OAEP-SHA256	Optimal Asymmetric Encryption Padding using SHA-256. A padding scheme applied to RSA encryption to prevent various attacks including chosen-ciphertext attacks. Used in the classical mode of PQC-AI for key transport.
RSA-2048	Rivest–Shamir–Adleman asymmetric encryption with a 2048-bit key. Security relies on the computational difficulty of factoring large integers on classical computers. Broken in polynomial time by Shor’s algorithm on a sufficiently large quantum computer.
Shor’s Algorithm	A quantum algorithm published by Peter Shor in 1994 that factors large integers in polynomial time. Directly breaks RSA, Diffie–Hellman, and Elliptic Curve Cryptography, motivating the development of post-quantum cryptographic standards.
TLS 1.3	Transport Layer Security version 1.3, the current standard for securing internet communications, defined in RFC 8446. The handshake protocol establishes session keys and authenticates parties before encrypted application data is exchanged.

Reflection of the Team Members on the Project

What we learned as a team:

As a team, we learned that building a system that sits at the intersection of cryptography, machine learning, and systems engineering requires far more integration work than any single domain alone. We discovered that theoretical knowledge of PQC algorithms is very different from operational knowledge — the challenges of key size management, correct FIPS-compliant validation steps, and the subtleties of authenticated encryption only became apparent during implementation. Coordinating across multiple modules (the RL engine, the inference layer, the cryptographic layer, and the API middleware) forced us to develop clear interface contracts and communication norms that improved our software engineering practice considerably.

What each member learned individually:

Every team member developed a deeper specialization in their core field and gained significant breadth of knowledge in fields associated with it. Interaction with liboqs and NIST FIPS standards provided exposure to cryptography-related tools used in professional practice, going beyond what would normally be covered in academic courses. Development and tuning of the Double Q-learning policy, especially concerning the experience replay memory and re-exploration process, provided useful lessons about online reinforcement learning under noisy conditions.

Strengths and weaknesses of the design process:

An important strength of the design is the modular, hierarchical structure, through which it was possible to work separately on each module (policy engine, inference core, cryptography layer), testing and benchmarking the individual components before integration due to well-defined interfaces between them. The above approach made debugging and profiling significantly easier. An important limitation related to the design is that all our experiments were conducted on a single machine using the CPU only, meaning that the performance characteristics may not be representative when running on a real GPU-accelerated setup with multiple workers. In this regard, it would have been better to establish a GPU baseline from the very beginning and to perform more thorough ablation experiments to assess the effect of different RL design choices (sensitivity to per-sensitivity tables, experience replay, DLASB).

Amit Kumar Kar

sec_4_grp_5_project_report

 Report/Manuscript

Document Details

Submission ID

trn:oid::3618:140261615

Submission Date

25 May 2026, 01:40 GMT+5:30

Download Date

25 May 2026, 01:46 GMT+5:30

File Name

sec_4_grp_5_project_report.docx

File Size

1.5 MB

67 Pages**13,737 Words****82,752 Characters**

12% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Submitted works

Match Groups

-  **87** Not Cited or Quoted 9%
Matches with neither in-text citation nor quotation marks
-  **35** Missing Quotations 3%
Matches that are still very similar to source material
-  **4** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
-  **0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 10%  Internet sources
- 8%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 87** Not Cited or Quoted 9%
Matches with neither in-text citation nor quotation marks
- 35** Missing Quotations 3%
Matches that are still very similar to source material
- 4** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
- 0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 10% Internet sources
- 8% Publications
- 0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
	www.coursehero.com	2%
2	Internet	
	www.simonsfoundation.org	1%
3	Internet	
	www.mdpi.com	1%
4	Internet	
	arxiv.org	<1%
5	Publication	
	Pasindu Udugahapattuwa, Chamitha de Alwis, Engin Zeydan, Uditha Wijewardha...	<1%
6	Publication	
	Cyprian Omukhwaya Sakwa, Fagen Li. "Survey on post-quantum cryptography im...	<1%
7	Internet	
	www.aiktdspace.org:8080	<1%
8	Publication	
	Akinlemi Olushola, S. P. Meenakshi. "Design and implementation of an authentic...	<1%
9	Internet	
	www.slideshare.net	<1%
10	Internet	
	www.quantumlogistics.com	<1%

11	Internet	i4c.semnan.ac.ir	<1%
12	Internet	studentsrepo.um.edu.my	<1%
13	Publication	Aasim Zafar, Syed Shamikh Iqbal. "Integrating code-based post-quantum cryptog...	<1%
14	Internet	www.irjet.net	<1%
15	Internet	instadevops.com	<1%
16	Publication	Bhupendra Singh, Mohankumar Mysamy, Thamaraimanalan Thangarajan. "Qua...	<1%
17	Internet	ijnrd.org	<1%
18	Internet	mdpi-res.com	<1%
19	Internet	www.preprints.org	<1%
20	Internet	github.com	<1%
21	Internet	sebastienrousseau.com	<1%
22	Internet	markaicode.com	<1%
23	Publication	Pushpa Choudhary, Sambit Satpathy, Arvind Dagur, Dharendra Kumar Shukla. "Re...	<1%
24	Internet	dev.to	<1%

25	Internet	ebin.pub	<1%
26	Internet	community.milkv.io	<1%
27	Internet	d-nb.info	<1%
28	Internet	todaynews.pro	<1%
29	Publication	Rong Wang, Bo Yuan, Minfu Yuan, Yin Li. "NTRU-MCF: A Chaos-Enhanced Multidi...	<1%
30	Publication	Vikas Chouhan, Mohammed Aldarwbi, Somayeh Sadeghi, Ali Ghorbani, Aaron Cho...	<1%
31	Internet	icsc2023-mlops.docs.cern.ch	<1%
32	Publication	Bullock, Monique. "Advanced Encryption for Secure Network Communication: A C...	<1%
33	Publication	Tiffany A. Ceasor. "Formal Decision Traces for Data-Driven Verification and Post- ...	<1%
34	Internet	uis.brage.unit.no	<1%
35	Publication	"Computer Security. ESORICS 2024 International Workshops", Springer Science an...	<1%
36	Publication	Jitendra Sharma, Manjari Rai, Udit Mamodiya, Gunja Soni, Aastha Saxena. "Emerg...	<1%
37	Publication	Mark D. Shermis, Joshua Wilson. "The Routledge International Handbook of Auto...	<1%
38	Internet	www.frontiersin.org	<1%

39	Publication	"Transformative Technologies Shaping a Smarter Society", Springer Science and ...	<1%
40	Publication	Alsanousi, Bassam Jameel. "Leveraging LLMs for the Analysis of Mobile App User ...	<1%
41	Publication	Chung-Yen Wu, Kuo-Hsuan Huang, Chih-Yao Hsu. "A Decentralised Multi-Authorit...	<1%
42	Publication	Tao Xia, Menglin Wang, Jun He, Gang Yang, Linna Fan, Guoheng Wei. "A Quantum...	<1%
43	Internet	aclanthology.org	<1%
44	Internet	computerfraudsecurity.com	<1%
45	Internet	docs.datarobot.com	<1%
46	Internet	ju.diva-portal.org	<1%
47	Internet	www.numberanalytics.com	<1%
48	Internet	zir.nsk.hr	<1%
49	Publication	"Quantum Protocols in Blockchain Security", Springer Science and Business Medi...	<1%
50	Publication	Fatima G. Abdullah, Tayseer S. Atia. "Post-Quantum Authentication in the Interne...	<1%
51	Publication	Mozamel M. Saeed, Fahad Alqahtani. "AI and post-quantum cryptography powere...	<1%
52	Publication	S Divya, R. Bagavathi Lakshmi. "Migration Strategies from Traditional Cryptograp...	<1%

53	Internet	bibliotecadigital.fgv.br	<1%
54	Internet	community.clearlinux.org	<1%
55	Internet	dspace.qou.edu	<1%
56	Internet	dspace.univ-boumerdes.dz	<1%
57	Internet	eprint.iacr.org	<1%
58	Internet	ir.aiktclibrary.org:8080	<1%
59	Internet	journals.indexcopernicus.com	<1%
60	Internet	pdf.usaid.gov	<1%
61	Internet	www.ijert.org	<1%
62	Internet	www.iris.unisa.it	<1%
63	Internet	www.researchgate.net	<1%
64	Internet	www.scifiniti.com	<1%
65	Internet	www.techscience.com	<1%
66	Publication	"Information and Communications Security", Springer Science and Business Medi...	<1%

67	Publication	Ankit Gupta, Shilpi Mittal. "Post-quantum readiness and cryptographic transition ...	<1%
68	Publication	Naya Nagy, Sarah Alnemer, Lama Mohammed Alshuhail, Haifa Alobiad et al. "Mo...	<1%
69	Publication	Sreeja Balachandran Nair Premakumari, Gopikrishnan Sundaram, Marco Rivera, ...	<1%
70	Publication	Yihong Zhu, Leibo Liu. "Survey of Post-Quantum Cryptographic Chips: Current Sta...	<1%
71	Publication	"Innovative Computing and Communications", Springer Science and Business Me...	<1%
72	Publication	Alex Khang. "AI-Powered Cybersecurity for Banking and Finance - How to Enhanc...	<1%
73	Publication	Wagner, Jonathan Armand. "Integrating Random Linear Code Based Encryption S...	<1%

